Chapter 2
Evasion Attacks

PCC190 – Adversarial Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 27, 2026

Contents

Notation **2**

1 Adversarial Examples and Evasion Attacks **2**
1.1 Overview . . . 2
1.2 Formal Definitions . . . 2

2 Problem Formulation **3**
2.1 Budget-Constrained Formulation . . . 3
2.2 Minimum-Norm Formulation . . . 4
2.3 Equivalence of the Two Formulations . . . 4
2.4 Bounds on Robust Accuracy . . . 5
2.5 General Threat Models . . . 5

3 Classification of Evasion Attacks **6**

4 White-Box Attacks **6**
4.1 Gradient-Based Budget-Constrained Attacks . . . 7
4.1.1 Fast Gradient Sign Method (FGSM) . . . 7
4.1.2 Basic Iterative Method (BIM) . . . 9
4.1.3 Projected Gradient Descent (PGD) . . . 9
4.2 Gradient-Based Minimum-Norm Attacks . . . 11
4.2.1 Carlini and Wagner . . . 11
4.2.2 DeepFool . . . 12

5 Query-Based Attacks **14**
5.1 Score-Based Attacks . . . 14
5.1.1 Gradient Estimation . . . 14
5.1.2 Random Search . . . 14
5.2 Decision-Based Attacks . . . 15

6 Transfer Attacks **15**

7 Ensembles of Models **16**

8 Large Language Models **17**
8.1 Threat Model . . . 17
8.2 Loss Function and Problem Formulation . . . 18
8.3 Greedy Coordinate Gradient . . . 19
8.4 Black-Box Attacks . . . 19
8.5 Prompt Injection . . . 20
8.6 Defenses . . . 20

9 Implementation **21**
9.1 FGSM and PGD . . . 21
9.2 Carlini–Wagner ℓ_2 . . . 22

9.3 NES with PGD . . . 22
9.4 Transfer from an Ensemble . . . 22
9.5 Suggested Experiments . . . 22

Exercises **23**

Notation

Model. We keep the notation of Chapter 1. The label set is $\mathcal{Y} = \{1, \dots, K\}$ and the input domain is $\mathcal{X} = [0,1]^d$. The classifier $f : \mathcal{X} \to \mathcal{Y}$ is $f(\mathbf{x}) = \arg\max_k Z_k(\mathbf{x})$, where $Z(\mathbf{x}) \in \mathbb{R}^K$ are the logits and $F(\mathbf{x}) = \mathrm{softmax}(Z(\mathbf{x}))$ the probabilities. The loss is $\mathcal{L}(\mathbf{x}, y)$, for example the cross-entropy $-\log F_y(\mathbf{x})$. We drop $\boldsymbol{\theta}$ from the notation because the parameters are fixed at inference time. $\mathcal{D}$ is the data distribution over $\mathcal{X} \times \mathcal{Y}$.

Clean point, candidate, perturbation. Throughout the chapter $\mathbf{x}$ is a clean input with label y, and we assume $f(\mathbf{x}) = y$ unless stated otherwise. Any other input, whether it is a candidate under evaluation or the output of an attack, is written $\mathbf{x}'$, and the corresponding perturbation is $\boldsymbol{\delta} = \mathbf{x}' - \mathbf{x}$; the two notations are interchangeable, $\mathbf{x}' = \mathbf{x} + \boldsymbol{\delta}$. Iterates of an algorithm are $\mathbf{x}^{(k)}$, with $\mathbf{x}^{(0)}$ the starting point, and $t \neq y$ denotes a target class.

Norms and threat balls. For $p \in [1, \infty)$, $\|\mathbf{v}\|_p = (\sum_i |v_i|^p)^{1/p}$, and $\|\mathbf{v}\|_\infty = \max_i |v_i|$. We also use $\|\mathbf{v}\|_0 = |\{i : v_i \neq 0\}|$, the number of modified coordinates, which is not a norm but is used as a perturbation size. For $p \geq 1$ the conjugate exponent q is defined by $1/p + 1/q = 1$ (with $q = 1$ for $p = \infty$ and $q = \infty$ for $p = 1$). The *threat ball* of radius $\epsilon \geq 0$ around $\mathbf{x}$ is

$$\mathcal{B}_p(\mathbf{x}, \epsilon) = \{\mathbf{x}' \in \mathbb{R}^d : \ \|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon\},$$

and the *admissible set* is its intersection with the input domain,

$$\mathcal{S}_p(\mathbf{x}, \epsilon) = \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}.$$

$\mathcal{B}_p(\mathbf{x}, \epsilon)$ is closed and, for $p \geq 1$, convex; it is not contained in $\mathcal{X}$, which is why the intersection is needed. For $p = \infty$, $\mathcal{S}_\infty(\mathbf{x}, \epsilon)$ is the box $\prod_i [\max(x_i - \epsilon, 0), \min(x_i + \epsilon, 1)]$. For images the budget ϵ is often quoted in 0–255 units, so $\epsilon = 8/255$ and "$\epsilon = 8$" denote the same ball.

Operators. $\Pi_S(\mathbf{z}) = \arg\min_{\mathbf{u} \in S} \|\mathbf{u} - \mathbf{z}\|_2$ is the Euclidean projection onto a closed convex set S, and $\mathrm{clip}_{[0,1]}(\mathbf{z})$ is the elementwise projection onto $\mathcal{X}$. $\mathbf{e}_i$ is the i-th canonical basis vector.

1 Adversarial Examples and Evasion Attacks

1.1 Overview

Evasion attacks craft inputs at inference time that cause a trained model to misbehave, without touching its weights or its training data. This chapter studies the algorithms that find such inputs. Two things describe the attacker's power: *which inputs* it may submit, the threat set, and *what it may observe* about the model, the information level.

The chapter is built around these two ingredients. This section turns them into definitions. Section 2 shows that finding an adversarial example amounts to solving one of two optimization problems, and Section 3 classifies attacks by the problem they solve, the information they use and the solver they apply. Sections 4 to 7 then present the attacks in order of decreasing information, from full access to the model to no access at all. Section 8 carries the same framework over to large language models, where the threat set is no longer a ball. Section 9 closes with reference implementations.

1.2 Formal Definitions

We state the definitions for the ℓ_p admissible set $\mathcal{S}_p(\mathbf{x}, \epsilon)$ introduced in the Notation section. Section 2.5 shows that they extend to other threat sets without change.

Definition 1.1 (Adversarial example)**.** Let $\mathbf{x} \in \mathcal{X}$ with $f(\mathbf{x}) = y$, let $p \geq 1$ and $\epsilon > 0$. A point $\mathbf{x}' \in \mathcal{X}$ is an *adversarial example* for $(\mathbf{x}, y)$ at budget ϵ if $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ and

$$\text{(untargeted)} \ \ f(\mathbf{x}') \neq y \qquad \text{or} \qquad \text{(targeted, } t \neq y\text{)} \ \ f(\mathbf{x}') = t.$$

Since f is an argmax, the untargeted condition is equivalent to $\max_{j\neq y} Z_j(\mathbf{x}') - Z_y(\mathbf{x}') \geq 0$ (up to tie-breaking), and the targeted one to $Z_t(\mathbf{x}') > \max_{j\neq t} Z_j(\mathbf{x}')$.

The definition is a property of a *point*; it says nothing about how the point was found. An attack is the procedure that finds it.

Definition 1.2 (Evasion attack)**.** An *evasion attack* is a (possibly randomized) algorithm $\mathcal{A}$ that receives $(\mathbf{x}, y)$ (and t if targeted), a norm p, possibly a budget ϵ, and access to f only through an information level $\mathcal{I} \in \{\text{gradients}, \text{scores}, \text{labels}, \text{none}\}$ (white-box, score-based, decision-based, transfer). It returns a point

$$\mathbf{x}' = \mathcal{A}^{\mathcal{I}(f)}(\mathbf{x}, y) \in \mathcal{X}.$$

A *budget attack* receives ϵ and guarantees $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$. A *min-norm attack* receives no budget and tries to return a misclassified $\mathbf{x}'$ with $\|\mathbf{x}' - \mathbf{x}\|_p$ as small as possible. The attack acts only on the input: $\boldsymbol{\theta}$ and the training set are fixed, and $\mathcal{A}$ is run at inference time. Level $\mathcal{I} = \text{none}$ means that no query to the target is made for the input being attacked; a surrogate model may still have been built beforehand, possibly by querying the target (Section 6).

Definition 1.3 (Attack success and robustness)**.** At budget ϵ, an attack *succeeds* on $(\mathbf{x}, y)$ if its output is an adversarial example in the sense of Definition 1.1, that is, $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ and $f(\mathbf{x}') \neq y$. The *attack success rate* is

$$\mathrm{ASR}(\mathcal{A}, \epsilon) = \Pr_{(\mathbf{x},y)\sim\mathcal{D}}\Big[\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon) \text{ and } f(\mathbf{x}') \neq y \;\Big|\; f(\mathbf{x}) = y\Big], \qquad \mathbf{x}' = \mathcal{A}^{\mathcal{I}(f)}(\mathbf{x}, y),$$

and the *robust accuracy* of f is

$$\mathrm{RA}(f, \epsilon) = \Pr_{(\mathbf{x},y)\sim\mathcal{D}}\Big[\forall\, \mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon) :\; f(\mathbf{x}') = y\Big].$$

For a budget attack the membership condition holds by construction. For a min-norm attack it is the check $\|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon$, so one run of a min-norm attack yields the success rate at every ϵ simultaneously. Note also that RA is a property of f alone, while ASR depends on the attack, and the gap between them is the subject of Section 2.4. Computing RA exactly would require checking every point of $\mathcal{S}_p(\mathbf{x}, \epsilon)$, which is impossible for a continuous set. What an attacker, or an evaluator playing the attacker, can do instead is search the set for a witness. The next section formulates that search.

2 Problem Formulation

Definition 1.3 asks a universal question, whether f is correct on every point of $\mathcal{S}_p(\mathbf{x}, \epsilon)$, and an attacker answers it negatively by exhibiting one witness. The search for a witness can be posed in two ways: fix the budget and look for the worst point inside it, or fix the goal, misclassification, and look for the cheapest point that achieves it. Every attack in this chapter, including the black-box and LLM attacks, solves one of these two problems exactly or approximately, which is why "optimization-based" cannot name a family of attacks. This section states both problems (Sections 2.1 and 2.2), proves that they describe the same object (Section 2.3), explains what can and cannot be computed about it (Section 2.4), and shows that nothing in the formulation depends on the threat set being an ℓ_p ball (Section 2.5).

2.1 Budget-Constrained Formulation

Definition 2.1 (Budget problem)**.** Let $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$, a norm $p \geq 1$ and a budget $\epsilon > 0$. The *budget problem* is

$$\max_{\mathbf{x}'\in\mathcal{S}_p(\mathbf{x},\epsilon)} \mathcal{L}(\mathbf{x}', y) \;=\; \max_{\substack{\|\boldsymbol{\delta}\|_p \leq \epsilon \\ \mathbf{x}+\boldsymbol{\delta}\in\mathcal{X}}} \mathcal{L}(\mathbf{x} + \boldsymbol{\delta}, y). \tag{1}$$

Its targeted version, for a target class $t \neq y$, is $\min_{\mathbf{x}'\in\mathcal{S}_p(\mathbf{x},\epsilon)} \mathcal{L}(\mathbf{x}', t)$, which is the same problem with $\mathcal{L}(\cdot, y)$ replaced by $-\mathcal{L}(\cdot, t)$.

The feasible set $\mathcal{S}_p(\mathbf{x}, \epsilon)$ is convex and compact, and for $p \in \{2, \infty\}$ the projection onto the ball is available in closed form (Section 4.1). The difficulty is entirely in the objective: $\mathcal{L}$ is nonconcave in $\mathbf{x}'$ for a neural network, so (1) has no closed-form solution and may have many local maxima. A budget attack answers the question "can the model be fooled within this budget", one value of ϵ at a time. The second formulation removes the budget from the problem and makes it the output.

2.2 Minimum-Norm Formulation

Definition 2.2 (Min-norm problem)**.** Let $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$ and a norm $p \geq 1$ (or $p = 0$). The *min-norm problem* is

$$\epsilon^\star(\mathbf{x}, y) \;=\; \inf_{\boldsymbol{\delta}} \; \|\boldsymbol{\delta}\|_p \quad \text{s.t.} \quad f(\mathbf{x}+\boldsymbol{\delta}) \neq y, \;\; \mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}. \tag{2}$$

Its targeted version replaces $f(\mathbf{x}+\boldsymbol{\delta}) \neq y$ by $f(\mathbf{x}+\boldsymbol{\delta}) = t$. The value $\epsilon^\star(\mathbf{x}, y)$ is the *minimal adversarial perturbation*, or distance to the decision boundary.

We write inf because the feasible set need not be closed (its boundary contains ties of the argmax); in the literature it is usually written as a min. Here the situation is reversed with respect to the budget problem: the objective $\|\boldsymbol{\delta}\|_p$ is convex, but the feasible set $\{\boldsymbol{\delta} : f(\mathbf{x}+\boldsymbol{\delta}) \neq y\} \cap (\mathcal{X} - \mathbf{x})$ is in general nonconvex, since it is the complement of the region where the model predicts y, and the constraint $f(\mathbf{x}+\boldsymbol{\delta}) \neq y$ is discrete, so it provides no gradient. A min-norm attack returns a distance, not a success rate at a fixed ϵ, and any feasible $\boldsymbol{\delta}$ it finds gives the upper bound $\epsilon^\star(\mathbf{x}, y) \leq \|\boldsymbol{\delta}\|_p$.

The two formulations look different, one with a convex feasible set and a nonconcave objective, the other with a convex objective and a nonconvex feasible set. They nevertheless carry the same information, as we show next.

2.3 Equivalence of the Two Formulations

Proposition 2.1 (Duality between the two problems)**.** *Fix $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$. If $\epsilon^\star(\mathbf{x}, y) < \epsilon$, an adversarial example at budget ϵ exists; if $\epsilon^\star(\mathbf{x}, y) > \epsilon$, none exists. Consequently*

$$\Pr_{(\mathbf{x},y)\sim\mathcal{D}}\big[\epsilon^\star(\mathbf{x}, y) > \epsilon\big] \;\leq\; \mathrm{RA}(f, \epsilon) \;\leq\; \Pr_{(\mathbf{x},y)\sim\mathcal{D}}\big[\epsilon^\star(\mathbf{x}, y) \geq \epsilon\big],$$

where $\epsilon^\star = 0$ by convention for misclassified points.

Proof. If $\epsilon^\star < \epsilon$, by definition of the infimum there is a feasible $\boldsymbol{\delta}$ with $\|\boldsymbol{\delta}\|_p < \epsilon$, and $\mathbf{x}+\boldsymbol{\delta}$ is an adversarial example at budget ϵ. If $\epsilon^\star > \epsilon$, every misclassified point of $\mathcal{X}$ is at distance larger than ϵ from $\mathbf{x}$, so f is constant on $\mathcal{S}_p(\mathbf{x}, \epsilon)$. The bounds on RA follow by taking probabilities. □

Up to the boundary case $\epsilon^\star = \epsilon$, the robust accuracy curve $\epsilon \mapsto \mathrm{RA}(f, \epsilon)$ is the survival function of the random variable $\epsilon^\star(\mathbf{x}, y)$. This is the precise sense in which the two problems are two views of the same object. A budget attack estimates one point of this curve; a min-norm attack, run once per input, estimates the whole curve, at the price of solving a harder problem per input.

Example 2.1 (A linear classifier)**.** Let $d = 2$, $K = 2$, and let the logits satisfy $Z_2(\mathbf{x}) - Z_1(\mathbf{x}) = m(\mathbf{x}) = \mathbf{a}^\top\mathbf{x} + b$ with $\mathbf{a} = (3, 4)$ and $b = -4$. Ties are broken in favor of class 2, so $f(\mathbf{x}) = 2$ if and only if $m(\mathbf{x}) \geq 0$, and the infimum in (2) is attained. The point $\mathbf{x} = (0.2, 0.6)$ has $m(\mathbf{x}) = -1$, so $f(\mathbf{x}) = 1 = y$, and a perturbation $\boldsymbol{\delta}$ is adversarial if and only if $\mathbf{a}^\top\boldsymbol{\delta} \geq 1$. Hölder's inequality gives $\mathbf{a}^\top\boldsymbol{\delta} \leq \|\mathbf{a}\|_q\|\boldsymbol{\delta}\|_p$, so every adversarial perturbation satisfies $\|\boldsymbol{\delta}\|_p \geq 1/\|\mathbf{a}\|_q$, with equality for the perturbation that makes Hölder's inequality tight (Proposition 4.1 below gives it explicitly). Since $\|\mathbf{a}\|_2 = 5$, $\|\mathbf{a}\|_1 = 7$ and $\|\mathbf{a}\|_\infty = 4$,

p	$\epsilon^\star(\mathbf{x}, y)$	$\boldsymbol{\delta}^\star$	$\mathbf{x}+\boldsymbol{\delta}^\star$
2	$1/5 = 0.2$	(0.12, 0.16)	(0.32, 0.76)
∞	$1/7 \approx 0.143$	(1/7, 1/7)	(0.343, 0.743)
1	$1/4 = 0.25$	(0, 0.25)	(0.2, 0.85)

All three points lie in $\mathcal{X}$, so the box constraint is inactive and the values are exact. The distance to the same boundary depends on the norm through the dual norm of $\mathbf{a}$. The ℓ_∞ minimizer moves every coordinate by the same amount, the ℓ_1 minimizer moves only the coordinate with the largest $|a_i|$, and the ℓ_2 minimizer moves each coordinate in proportion to a_i. Because $\|\boldsymbol{\delta}\|_\infty \leq \|\boldsymbol{\delta}\|_2 \leq \|\boldsymbol{\delta}\|_1$ for every $\boldsymbol{\delta}$, the ordering $\epsilon^\star_\infty \leq \epsilon^\star_2 \leq \epsilon^\star_1$ holds for every model and every input, not only here. By Proposition 2.1, the point is robust at ℓ_∞ budget ϵ if and only if $\epsilon < 1/7$, which is about $36/255$; the common budget $8/255$ leaves it robust. Sections 4 to 6 run their attacks on this example.

The two problems are also linked algorithmically. Replacing the discrete constraint of (2) by a penalty gives

$$\min_{\boldsymbol{\delta}} \; \|\boldsymbol{\delta}\|_p + c \cdot h(\mathbf{x}+\boldsymbol{\delta}), \tag{3}$$

where $h \leq 0$ implies misclassification and $c > 0$ trades size for success. For large c the penalty dominates and the problem behaves like a search for misclassification; for small c it behaves like norm minimization. Carlini and Wagner (Section 4.2.1) solve (3) and search over c, and the same function h (a logit margin) is often used as the objective of the budget problem instead of $\mathcal{L}$.

Remark 2.1 (The surrogate gap)**.** The quantity the attacker actually cares about is the indicator $\mathbb{1}[f(\mathbf{x}') \neq y]$, which has zero gradient almost everywhere. The budget problem replaces it by the surrogate $\mathcal{L}(\mathbf{x}', y)$ (or a logit margin), and the penalty form (3) replaces the constraint by h. Hence an attack that maximizes the surrogate need not find an adversarial example even when one exists, and every candidate must be checked with f. The attacks below differ in surrogate, solver and information level, and much of the practical advice in this chapter (margin instead of cross-entropy, random restarts, confidence κ) is about narrowing this gap.

2.4 Bounds on Robust Accuracy

Proposition 2.1 relates the two problems exactly, but for a neural network neither can be solved exactly. The robust accuracy $\mathrm{RA}(f, \epsilon)$ contains a universal quantifier over a continuous set, so it is not computable by enumeration. Deciding whether an adversarial example exists, equivalently whether $\epsilon^\star(\mathbf{x}, y) \leq \epsilon$, is a nonconvex problem, and for ReLU networks it is NP-hard in general (**?**). Exact verifiers exist (SMT or mixed-integer programming based), but they scale only to small networks. In practice one brackets RA, or $\epsilon^\star$, from both sides.

Upper bound (attacks). Any attack $\mathcal{A}$ that returns $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ exhibits a witness whenever $f(\mathbf{x}') \neq y$. Hence

$$\mathrm{RA}(f, \epsilon) \;\leq\; \mathrm{RA}_{\mathcal{A}}(f, \epsilon) \;=\; \Pr_{(\mathbf{x},y)\sim\mathcal{D}}\big[\mathcal{A} \text{ does not succeed on } (\mathbf{x}, y) \text{ at budget } \epsilon\big].$$

A stronger attack lowers this bound, a weaker one (FGSM, an attack defeated by gradient masking, too few iterations) leaves it loose. A *failed* attack is never a proof of robustness. Equivalently, every successful min-norm attack gives $\epsilon^\star(\mathbf{x}, y) \leq \|\mathbf{x}' - \mathbf{x}\|_p$.

Lower bound (certification). A certification method returns, for each input, a radius $r(\mathbf{x}, y)$ such that f is provably constant on $\mathcal{S}_p(\mathbf{x}, r)$, that is, $r \leq \epsilon^\star(\mathbf{x}, y)$. Counting the points with $r(\mathbf{x}, y) \geq \epsilon$ and $f(\mathbf{x}) = y$ gives

$$\mathrm{RA}_{\mathrm{cert}}(f, \epsilon) \;\leq\; \mathrm{RA}(f, \epsilon).$$

Typical routes are relaxation-based verification and verified training (**??**), which is deterministic but loose for large networks, and randomized smoothing (**?**), which certifies a *smoothed* classifier g, not f itself, with a probabilistic guarantee for ℓ_2.

Consequence. The true value lies in the interval

$$\mathrm{RA}_{\mathrm{cert}}(f, \epsilon) \;\leq\; \mathrm{RA}(f, \epsilon) \;\leq\; \mathrm{RA}_{\mathcal{A}}(f, \epsilon),$$

and the width of the interval measures how much we do not know. A narrow gap is only achieved for small models or models trained for verifiability. For standard deep networks the certified lower bound is typically near zero while the attack upper bound is not, so reported numbers must state which side they bound. Two further caveats: all three quantities are estimated on a finite test set, so they carry sampling error in addition to the gap above, and the attack bound holds only if the returned point is actually checked to satisfy $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ (see the implementation pitfalls).

2.5 General Threat Models

Everything so far was stated for the ℓ_p admissible set $\mathcal{S}_p(\mathbf{x}, \epsilon)$. Nothing in the two problems, however, depends on the set being an ℓ_p ball. Let $\mathcal{T}_\epsilon(\mathbf{x}) \subseteq \mathcal{X}$ be any family of admissible sets indexed by a budget, nested ($\mathcal{T}_\epsilon(\mathbf{x}) \subseteq \mathcal{T}_{\epsilon'}(\mathbf{x})$ for $\epsilon \leq \epsilon'$) and containing $\mathbf{x}$, or equivalently a cost $c(\mathbf{x}, \mathbf{x}') \geq 0$ with $\mathcal{T}_\epsilon(\mathbf{x}) = \{\mathbf{x}' : c(\mathbf{x}, \mathbf{x}') \leq \epsilon\}$. The two problems become

$$\text{(budget)} \quad \max_{\mathbf{x}' \in \mathcal{T}_\epsilon(\mathbf{x})} \mathcal{L}(\mathbf{x}', y), \qquad \text{(min-cost)} \quad \inf_{\mathbf{x}' \in \mathcal{X}} c(\mathbf{x}, \mathbf{x}') \;\text{ s.t. }\; f(\mathbf{x}') \neq y,$$

and Definitions 1.1–1.3, Proposition 2.1 and the bounds of Section 2.4 carry over verbatim with $\mathcal{S}_p(\mathbf{x}, \epsilon)$ replaced by $\mathcal{T}_\epsilon(\mathbf{x})$ and $\|\boldsymbol{\delta}\|_p$ by $c(\mathbf{x}, \mathbf{x}')$. The ℓ_p ball is the instance $c(\mathbf{x}, \mathbf{x}') = \|\mathbf{x}' - \mathbf{x}\|_p$. Other instances in use:

Why, then, does the chapter use ℓ_p balls for everything up to Section 8? Because the ball buys three things that the other sets lose to varying degrees: a projection in closed form (Section 4.1), a steepest ascent direction in closed form (Proposition 4.1), and certification methods with provable radii (Section 2.4). Spatial and Wasserstein sets keep a continuous structure and are attacked with the same first-order methods after reparametrization; patch and physical sets keep gradients but change the objective (an expectation over τ); problem-space and token sets are discrete and need search, as Section 8 shows.

Threat model	Admissible set $\mathcal{T}_\epsilon(\mathbf{x})$	Budget or cost
ℓ_p perturbation	$\mathcal{S}_p(\mathbf{x}, \epsilon)$	$\|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon$
Spatial transformation	$\{\tau_{\boldsymbol{\phi}}(\mathbf{x}) : \boldsymbol{\phi} \in \Phi_\epsilon\}$, rotations and translations (**?**)	range of angles and shifts
Wasserstein	$\{\mathbf{x}' : W(\mathbf{x}, \mathbf{x}') \leq \epsilon\}$, pixel mass moved spatially (**?**)	transport cost
Patch	$\{(\mathbf{1} - \mathbf{m}) \odot \mathbf{x} + \mathbf{m} \odot \mathbf{p} : \mathbf{p} \in \mathcal{X}\}$, mask $\mathbf{m}$ of fixed area (**?**)	patch area; no bound on $\mathbf{p}$
Physical world	$\{\tau(\mathbf{x}') : \tau \sim$ printing, viewpoint, lighting$\}$, success in expectation (**?**)	as for the digital set, averaged over τ
Problem space (malware, code)	outputs of functionality-preserving transformations (**?**)	number or type of transformations
Token sequences (LLMs)	suffixes, rewrites, injected documents (Section 8)	suffix length, access

Table 1: The budget and min-cost problems under different threat sets.

What ℓ_p results mean

The ℓ_p ball is a convenient, analyzable proxy for "imperceptible", not a model of what a real adversary can do. A small ℓ_p distance is neither necessary nor sufficient for an input to be "the same" to a human: a one-pixel shift of the whole image has large ℓ_p distance and is invisible, and a perturbation concentrated on a few pixels can have small ℓ_2 norm and be obvious. Every robustness statement is a statement about the chosen threat set, and robustness under one set, norm or budget does not imply robustness under another.

3 Classification of Evasion Attacks

Section 2 fixed what an attack computes. Attacks that compute the same thing can still differ widely in what they need from the model and in how they search, and this section organizes them along three largely independent axes.

Information level $\mathcal{I}$. What the attacker may observe about f (Definition 1.2). This is the axis that decides what is computable, and it is the top-level structure of the rest of the chapter: white-box attacks (Section 4), query-based black-box attacks with scores or labels (Section 5), and transfer-based black-box attacks (Section 6).

Problem solved. The budget problem (Definition 2.1) or the min-norm problem (Definition 2.2), possibly in the penalty form (3). Each of the following sections opens with a *Formulation* paragraph that states which problem its attacks solve and why the information level favors one or the other.

Solver. How the optimization is carried out: projected first-order steps, penalty methods with an adaptive optimizer, iterative linearization, zeroth-order gradient estimation, random search, or boundary random walks.

Table 2 places the attacks of this chapter on these axes. The partition is by information level only; the other two columns cut across it. For example, C&W and PGD are both white-box gradient methods and differ in the problem solved, whereas PGD and NES-PGD solve the same problem and differ in the information level, and therefore in how the gradient is obtained.

Two caveats on the classification. First, transfer is a strategy rather than an algorithm: it chooses where the optimization is run (on a surrogate), not how. Second, some attacks straddle categories: the margin loss of C&W is reused inside PGD (budget problem), and NES also covers the partial-information and label-only settings (**?**). The remaining sections follow the information axis from the most informed attacker to the least informed, starting with full access to the model.

4 White-Box Attacks

With $\mathcal{I}$ = gradients the attacker knows the architecture and the weights $\boldsymbol{\theta}$, and can compute derivatives of the model outputs with respect to the *input*. Concretely, for any $\mathbf{x}'$ the attacker can evaluate the logits $Z(\mathbf{x}') \in \mathbb{R}^K$

Attack	Information	Problem	Solver	Norms
FGSM	gradients	budget	one linearized step	ℓ_∞, ℓ_2
BIM, PGD	gradients	budget	projected first-order ascent	ℓ_∞, ℓ_2
APGD	gradients	budget	PGD with adaptive step and momentum	ℓ_∞, ℓ_2
C&W	gradients	min-norm (penalty)	margin penalty, Adam, binary search on c	ℓ_0, ℓ_2, ℓ_∞
DeepFool	gradients	min-norm	iterative linearization of the boundaries	ℓ_2
ZOO	scores	min-norm (penalty)	coordinate-wise finite differences	ℓ_2
NES, SPSA	scores	budget	zeroth-order gradient estimate inside PGD	ℓ_∞
Square Attack	scores	budget	random search on the ball vertices	ℓ_∞
Boundary Attack	labels	min-norm	random walk along the boundary	ℓ_2
HopSkipJump	labels	min-norm	estimated boundary normal and binary search	ℓ_2, ℓ_∞
Transfer	none (per input)	either, on the surrogate	any white-box solver on a surrogate	any

Table 2: Attacks of this chapter along the three axes. Entries are a summary of the cited papers and some attacks have variants outside the listed norms.

and, by backpropagation, the gradients

$$\nabla_{\mathbf{x}} Z_k(\mathbf{x}') \in \mathbb{R}^d \quad (k = 1, \dots, K), \qquad \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}', y) \in \mathbb{R}^d,$$

at the cost of one forward and one backward pass per gradient. The gradient is taken with respect to the input and not with respect to $\boldsymbol{\theta}$: the parameters are fixed at inference time (Definition 1.2), so $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ plays no role here, in contrast with training. For a ReLU network the logits are piecewise linear in the input, so these gradients exist almost everywhere, but they may be zero or uninformative (saturated softmax, gradient masking), a point we return to when evaluating defenses.

Formulation. Gradients serve both problems of Section 2. Attacks on the budget problem (1) ascend the surrogate using $\nabla_{\mathbf{x}} \mathcal{L}$, and the convex feasible set is handled by projection. Attacks on the min-norm problem (2) use the gradients of the logits, or of the margins $m_k = Z_k - Z_y$, either to build the penalty (3) or to locate and cross the nearest decision boundary. We first treat the budget problem and then the min-norm problem.

4.1 Gradient-Based Budget-Constrained Attacks

We begin with the budget problem (1). The three attacks of this subsection are consecutive refinements of one idea, and we derive each from the previous one:

- **FGSM (Fast Gradient Sign Method)** replaces $\mathcal{L}$ by its first-order linearization at $\mathbf{x}$ and solves the resulting linear problem exactly, in one step.
- **BIM (Basic Iterative Method)** repeats this step with a small step size $\alpha < \epsilon$, re-linearizing at every iterate because the linear model is only trustworthy locally, and maps each iterate back to $\mathcal{S}_p(\mathbf{x}, \epsilon)$ by projection.
- **PGD (Projected Gradient Descent)** formalizes the repetition as projected steepest ascent for any ℓ_p norm, and adds random starts and restarts to cope with the nonconcavity of (1).

4.1.1 Fast Gradient Sign Method (FGSM)

Step 1: a local linear model of the loss. Let $\mathbf{g} = \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y)$. A first-order Taylor expansion around $\mathbf{x}$ gives

$$\mathcal{L}(\mathbf{x} + \boldsymbol{\delta}, y) = \mathcal{L}(\mathbf{x}, y) + \boldsymbol{\delta}^\top \mathbf{g} + R(\boldsymbol{\delta}), \tag{4}$$

where the remainder $R(\boldsymbol{\delta})$ is of order $\|\boldsymbol{\delta}\|^2$ when $\mathcal{L}$ is twice differentiable. For a ReLU network the logits are piecewise linear in the input, so the linear model is exact inside the activation region that contains $\mathbf{x}$; the ball $\mathcal{B}_p(\mathbf{x}, \epsilon)$ may, however, cross many regions. Dropping R and the constant $\mathcal{L}(\mathbf{x}, y)$, and ignoring the box $\mathcal{X}$ for the moment, problem (1) becomes

$$\max_{\|\boldsymbol{\delta}\|_p \le \epsilon} \boldsymbol{\delta}^\top \mathbf{g}. \tag{5}$$

This is the problem of finding the perturbation of size at most ϵ that is best aligned with $\mathbf{g}$, which depends on how size is measured.

Step 2: solving the linear problem.

Proposition 4.1 (Steepest ascent direction)**.** *For any* $\mathbf{g} \in \mathbb{R}^d$, $\mathbf{g} \neq \mathbf{0}$, *and* $p \geq 1$ *with conjugate* q,

$$\max_{\|\boldsymbol{\delta}\|_p \leq \epsilon} \boldsymbol{\delta}^\top \mathbf{g} = \epsilon \|\mathbf{g}\|_q.$$

For $p = \infty$ *(*$q = 1$*) the maximum is attained at* $\boldsymbol{\delta} = \epsilon \operatorname{sign}(\mathbf{g})$*; for* $p = 2$ *(*$q = 2$*) at* $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$*; for* $1 < p < \infty$ *at* $\delta_i = \epsilon \operatorname{sign}(g_i)|g_i|^{q-1}/\|\mathbf{g}\|_q^{q-1}$.

Proof. Upper bound. Hölder's inequality states $\boldsymbol{\delta}^\top \mathbf{g} \leq \|\boldsymbol{\delta}\|_p \|\mathbf{g}\|_q$ for conjugate exponents $p, q \in [1, \infty]$. Hence $\boldsymbol{\delta}^\top \mathbf{g} \leq \epsilon \|\mathbf{g}\|_q$ for every feasible $\boldsymbol{\delta}$.

Attainment for $p = \infty$. Take $\boldsymbol{\delta} = \epsilon \operatorname{sign}(\mathbf{g})$. Then $\|\boldsymbol{\delta}\|_\infty = \epsilon$ and $\boldsymbol{\delta}^\top \mathbf{g} = \epsilon \sum_i |g_i| = \epsilon \|\mathbf{g}\|_1$, so the bound is tight.

Attainment for $1 < p < \infty$. Equality in Hölder's inequality requires $|\delta_i|^p$ proportional to $|g_i|^q$ with matching signs, which suggests $\delta_i = c \operatorname{sign}(g_i)|g_i|^{q-1}$. From $1/p + 1/q = 1$ we get $(q-1)p = q$. Therefore $\|\boldsymbol{\delta}\|_p^p = c^p \sum_i |g_i|^{(q-1)p} = c^p \|\mathbf{g}\|_q^q$, and $\|\boldsymbol{\delta}\|_p = \epsilon$ fixes $c = \epsilon/\|\mathbf{g}\|_q^{q/p} = \epsilon/\|\mathbf{g}\|_q^{q-1}$. The value is then $\boldsymbol{\delta}^\top \mathbf{g} = c \sum_i |g_i|^q = \epsilon \|\mathbf{g}\|_q^q / \|\mathbf{g}\|_q^{q-1} = \epsilon \|\mathbf{g}\|_q$. For $p = 2$ we have $q = 2$ and $c = \epsilon/\|\mathbf{g}\|_2$, which gives $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$. □

The case $p = 1$ ($q = \infty$) is Exercise **1.**. Two consequences are worth stating explicitly.

- For $p = \infty$ the optimal perturbation uses only the *sign* of each partial derivative, and the first-order gain is $\epsilon \|\mathbf{g}\|_1$. This sum has d terms, so for a fixed per-pixel budget ϵ the first-order gain grows with the input dimension. This is the linearity argument of **?**: many individually negligible changes add up. It explains why the effect exists at first order, not that the linear model remains accurate.
- For the targeted attack we want to *decrease* the loss of the target class. Maximizing $-\mathcal{L}(\mathbf{x}, t)$ is the same problem with $\mathbf{g}$ replaced by $-\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, t)$, so the proposition gives $\boldsymbol{\delta} = -\epsilon \operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, t))$.

Step 3: the box constraint. Problem (5) ignored $\mathbf{x} + \boldsymbol{\delta} \in \mathcal{X}$. For $p = \infty$ the constraints are separable: coordinate i must satisfy $\delta_i \in [l_i, u_i]$ with $l_i = -\min(\epsilon, x_i) \leq 0 \leq u_i = \min(\epsilon, 1 - x_i)$. The objective $\sum_i g_i \delta_i$ is linear in each coordinate, so the maximum sits at u_i if $g_i > 0$ and at l_i if $g_i < 0$. Since $x_i + u_i = \min(x_i + \epsilon, 1)$ and $x_i + l_i = \max(x_i - \epsilon, 0)$, this is exactly the sign step followed by clipping, so clipping loses nothing for ℓ_∞. The result is the attack of **?**,

$$\mathbf{x}' = \operatorname{clip}_{[0,1]}\big(\mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y))\big). \tag{6}$$

For $p = 2$ the same reasoning with Proposition 4.1 gives the *fast gradient method* (FGM), $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$; here clipping is only an approximation of the box-constrained optimum, because the constraints no longer separate.

Example 4.1 (FGSM on the linear classifier)**.** Return to Example 2.1 with the cross-entropy loss. With two classes, $\mathcal{L}(\mathbf{x}, 1) = \log\big(1 + e^{Z_2(\mathbf{x}) - Z_1(\mathbf{x})}\big) = \log\big(1 + e^{m(\mathbf{x})}\big)$, so $\nabla_{\mathbf{x}} \mathcal{L} = \sigma(m(\mathbf{x}))\, \mathbf{a}$, where σ is the logistic function, and $\operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}) = \operatorname{sign}(\mathbf{a}) = (1, 1)$ at every point. FGSM returns $\mathbf{x}' = \operatorname{clip}_{[0,1]}\big((0.2 + \epsilon,\ 0.6 + \epsilon)\big)$. For $\epsilon \leq 0.4$ nothing is clipped and $m(\mathbf{x}') = -1 + 7\epsilon$, so FGSM succeeds if and only if $\epsilon \geq 1/7 = \epsilon_\infty^\star$. For $\epsilon > 0.4$ the second coordinate is clipped to 1, $m(\mathbf{x}') = 0.6 + 3\epsilon > 0$, and FGSM still succeeds. FGSM therefore solves the budget problem exactly here. The reason is that the loss is an increasing function of the linear score m, so the maximizer of m over $\mathcal{S}_\infty(\mathbf{x}, \epsilon)$, which is the sign step followed by clipping (Step 3), also maximizes $\mathcal{L}$. The argument applies to any model whose loss is an increasing function of a linear score, and it breaks as soon as the score is nonlinear, as Example 4.2 shows.

When can one linear step be trusted? Suppose $\nabla_{\mathbf{x}} \mathcal{L}$ is β-Lipschitz in the Euclidean norm. Then, writing $\mathbf{x}_s = \mathbf{x} + s\boldsymbol{\delta}$,

$$\mathcal{L}(\mathbf{x} + \boldsymbol{\delta}) - \mathcal{L}(\mathbf{x}) - \boldsymbol{\delta}^\top \mathbf{g} = \int_0^1 \big(\nabla \mathcal{L}(\mathbf{x}_s) - \nabla \mathcal{L}(\mathbf{x})\big)^\top \boldsymbol{\delta}\, ds \ \geq\ -\int_0^1 \beta s \|\boldsymbol{\delta}\|_2^2\, ds = -\tfrac{\beta}{2}\|\boldsymbol{\delta}\|_2^2.$$

For the FGSM perturbation $\|\boldsymbol{\delta}\|_2^2 = \epsilon^2 d$, so the actual loss increase is at least $\epsilon\|\mathbf{g}\|_1 - \frac{\beta}{2}\epsilon^2 d$. This guarantee is positive only if $\epsilon < 2\|\mathbf{g}\|_1/(\beta d)$: the single-step guarantee degrades with the curvature β and with the dimension. It is a lower bound, not a prediction, and deep networks are not globally smooth, so it should be read as intuition. It explains two observations at once. A model whose loss surface is highly curved near the data (large β, as in the gradient masking induced by FGSM training) makes the single step uninformative, and a smaller step that is re-linearized, as in BIM below, gives guarantees at a much smaller ϵ.

Analysis. FGSM costs one forward and one backward pass. It is exactly optimal only if the loss is linear in the ball, which it is not for deep networks. Its value today is as a fast baseline and as an inexpensive generator for adversarial training. It is a *weak* attack for evaluation: a model can resist FGSM while being completely broken by iterative attacks. In fact, models trained only on FGSM examples are known to exhibit *label leaking* and a form of gradient masking where the loss surface becomes highly curved near the data, making the single linear step uninformative (**?**).

4.1.2 Basic Iterative Method (BIM)

The remedy suggested by the previous analysis is to take a step small enough for the linear model to hold, move, and linearize again. Let $\mathbf{x}^{(k)}$ be the current iterate and $\mathbf{g}^{(k)} = \nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)$. Within a trust region $\|\boldsymbol{\delta}\|_\infty \le \alpha$ around $\mathbf{x}^{(k)}$, Proposition 4.1 says that the best step for the local linear model is $\alpha\ \mathrm{sign}(\mathbf{g}^{(k)})$, that is, an FGSM step with budget $\alpha < \epsilon$. After the step the iterate may have left $\mathcal{S}_\infty(\mathbf{x}, \epsilon)$, which is not allowed, so we map it back with the projection $\Pi_{\mathcal{S}_\infty(\mathbf{x},\epsilon)}$, derived explicitly in the next subsection. This is the update of **?**:

$$\mathbf{x}^{(0)} = \mathbf{x}, \qquad \mathbf{x}^{(k+1)} = \Pi_{\mathcal{S}_\infty(\mathbf{x},\epsilon)}\Big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}\left(\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)\right)\Big). \tag{7}$$

How many steps? Every step changes each coordinate by at most α, so after k steps $\|\mathbf{x}^{(k)} - \mathbf{x}\|_\infty \le k\alpha$. Reaching the boundary of the ball therefore requires $T \ge \epsilon/\alpha$ iterations. With ϵ in 0–255 units and $\alpha = 1$ this reads $T \ge \epsilon$. Kurakin et al. use the heuristic $T = \lceil \min(\epsilon + 4,\ 1.25\epsilon) \rceil$, which adds slack to this lower bound. Our reading of the slack is that coordinates whose gradient sign flips between iterations move back, so more than ϵ/α steps are needed in practice, while the cap $\epsilon + 4$ keeps the cost bounded for large ϵ. For $\epsilon = 8$ the minimum is 8 steps and the heuristic gives $\lceil \min(12, 10) \rceil = 10$.

4.1.3 Projected Gradient Descent (PGD)

BIM fixes three choices: the ℓ_∞ norm, the sign of the gradient as step direction, and the clean input $\mathbf{x}$ as starting point. PGD keeps the idea and relaxes all three.

Projected steepest ascent. Let $S = \mathcal{S}_p(\mathbf{x}, \epsilon)$ be the feasible set of (1). The generic method for maximizing a differentiable function over a closed convex set is: move along an ascent direction, then return to the set with Π_S. The direction is a design choice. Plain gradient ascent uses $\mathbf{g}$; steepest ascent with respect to the ℓ_p norm uses the maximizer of $\mathbf{d}^\top \mathbf{g}$ over $\|\mathbf{d}\|_p \le 1$, which Proposition 4.1 gives in closed form. For the two norms of this chapter,

$$\mathbf{x}^{(k+1)} = \Pi_S\big(\mathbf{x}^{(k)} + \alpha\, \mathbf{d}^{(k)}\big), \qquad \mathbf{d}^{(k)} = \begin{cases} \mathrm{sign}(\mathbf{g}^{(k)}) & p = \infty, \\ \mathbf{g}^{(k)} / \|\mathbf{g}^{(k)}\|_2 & p = 2. \end{cases}$$

BIM is this iteration for $p = \infty$. What **?** add is the *starting point*, discussed below.

The projections. Both are short derivations.

Proposition 4.2 (Projection for ℓ_∞)**.** *Let* $S = \mathcal{S}_\infty(\mathbf{x}, \epsilon)$. *Then* $\Pi_S(\mathbf{z})_i = \min\big(\max(z_i,\ x_i - \epsilon,\ 0),\ x_i + \epsilon,\ 1\big)$.

Proof. The set S is the box $\{\mathbf{u} :\ a_i \le u_i \le b_i\}$ with $a_i = \max(x_i - \epsilon, 0)$ and $b_i = \min(x_i + \epsilon, 1)$, because the intersection of two boxes is a box. The objective $\|\mathbf{u} - \mathbf{z}\|_2^2 = \sum_i (u_i - z_i)^2$ is a sum of terms that each involve one coordinate, and the constraints also involve one coordinate each, so the problem splits into d scalar problems $\min_{a_i \le u_i \le b_i} (u_i - z_i)^2$. The minimizer of a convex parabola centered at z_i over an interval is z_i if it lies in the interval and the nearest endpoint otherwise, which is $\min(\max(z_i, a_i), b_i)$. □

Proposition 4.3 (Projection onto an ℓ_2 ball)**.** *For* $S = \mathcal{B}_2(\mathbf{x}, \epsilon)$, $\Pi_S(\mathbf{z}) = \mathbf{x} + \epsilon\, \mathbf{v} / \max(\epsilon, \|\mathbf{v}\|_2)$ *with* $\mathbf{v} = \mathbf{z} - \mathbf{x}$.

Proof. Write $\mathbf{u} = \mathbf{x} + \mathbf{w}$ with $\|\mathbf{w}\|_2 \le \epsilon$, so the problem is $\min \|\mathbf{w} - \mathbf{v}\|_2$. If $\|\mathbf{v}\|_2 \le \epsilon$, then $\mathbf{w} = \mathbf{v}$ is feasible with value zero. Otherwise, the triangle inequality gives $\|\mathbf{w} - \mathbf{v}\|_2 \ge \|\mathbf{v}\|_2 - \|\mathbf{w}\|_2 \ge \|\mathbf{v}\|_2 - \epsilon$, with equality exactly when $\mathbf{w}$ points in the direction of $\mathbf{v}$ and $\|\mathbf{w}\|_2 = \epsilon$, that is, $\mathbf{w} = \epsilon\mathbf{v}/\|\mathbf{v}\|_2$. □

For ℓ_2 the feasible set $\mathcal{S}_2(\mathbf{x}, \epsilon)$ also includes the box, and the intersection of a ball and a box is not a box, so the problem no longer splits. Rescaling and then clipping is therefore only an approximation of Π_S, which is the usual practice (Exercise **2.**).

Why random starts and restarts. Problem (1) is nonconcave, so the iteration converges (if at all) to a local maximum determined by its starting point. Starting from $\mathbf{x}$ itself is a deterministic choice: if the gradient there is uninformative, for example because cross-entropy saturates on a very confident input, every restart from $\mathbf{x}$ fails in the same way. Sampling the start $\mathbf{x}^{(0)} = \mathrm{clip}_{[0,1]}(\mathbf{x} + \mathbf{u})$, $\mathbf{u} \sim \mathcal{U}[-\epsilon, \epsilon]^d$, places the iterate in different basins of attraction. Running R restarts and keeping the best of them gives a maximum over R local maxima, which cannot decrease as R grows. This is the reason for the sanity check below that more restarts should never raise the reported accuracy.

Example 4.2 (Why one step and one start are not enough)**.** We write scalars in plain type. Let $d = 1$, $\mathcal{X} = [0, 1]$, two classes, and margin

$$m(x) = \max\big(6(0.35 - x),\ x - 0.7\big),$$

the maximum of two affine functions, which a small ReLU network represents exactly, since $\max(u, v) = v + \max(u - v, 0)$. With ties going to class 2, the model predicts class 1 exactly on $(0.35, 0.7)$. Take $x = 0.5$, $y = 1$, budget $\epsilon = 0.18$, and any loss that is increasing in m, so that the sign of the loss gradient is the sign of m'. The two pieces are equal at $x = 0.4$. On $(0.4, 1]$ the second piece is active, with slope $+1$; on $[0, 0.4)$ the first piece is active, with slope -6. The nearest boundary is at 0.35, so $\epsilon^\star = 0.15 < \epsilon$ and an adversarial example exists at this budget; the other boundary, at 0.7, is at distance $0.2 > \epsilon$.

FGSM. At $x = 0.5$ the slope is $+1$, so FGSM returns $x' = 0.68$ with $m(0.68) = -0.02 < 0$ and fails. The gradient points towards the far boundary because the steep piece, which leads to the near one, is not active at x.

PGD started at x. With $\alpha = 0.02$, every iterate stays in $(0.4, 0.68]$, where the slope is $+1$, so the iteration moves right and stops at the edge of the ball, 0.68. More iterations do not help.

PGD with a random start. Let $x^{(0)}$ be uniform on $[0.32, 0.68]$. If $x^{(0)} < 0.4$, the slope is -6, the iterates move left and reach 0.32 in at most four steps, and $m(0.32) = 0.18 > 0$: the attack succeeds. If $x^{(0)} > 0.4$, it fails as before. A single restart therefore succeeds with probability $0.08/0.36 = 2/9$, and R independent restarts succeed with probability $1 - (7/9)^R$, about 0.92 for $R = 10$.

The failures are not caused by too few iterations or too small a step: the gradient at x points to the wrong basin. Random restarts are the remedy PGD offers, and they work only in probability. A min-norm attack started at x runs into the same problem (Example 4.4).

Algorithm 1 PGD attack, ℓ_∞, untargeted

Require: $\mathbf{x}, y$, budget ϵ, step size α, iterations T, restarts R

$\mathbf{x}'_{\text{best}} \leftarrow \mathbf{x}$
for $r = 1, \dots, R$ **do**
  $\mathbf{x}^{(0)} \leftarrow \mathrm{clip}_{[0,1]}(\mathbf{x} + \mathbf{u})$, $\mathbf{u} \sim \mathcal{U}[-\epsilon, \epsilon]^d$
  for $k = 0, \dots, T - 1$ **do**
    $\mathbf{g} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y)$
    $\mathbf{x}^{(k+1)} \leftarrow \Pi_{\mathcal{S}_\infty(\mathbf{x}, \epsilon)}\big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}(\mathbf{g})\big)$
  end for
  keep $\mathbf{x}^{(T)}$ as $\mathbf{x}'_{\text{best}}$ if it misclassifies (or has higher loss)
end for
return $\mathbf{x}'_{\text{best}}$

Why PGD became the reference first-order attack. **?** observed empirically that, from many random starts, PGD finds local maxima of the loss with remarkably concentrated values. This suggests that, for first-order methods, PGD is close to the best one can do, which motivated using PGD both to evaluate models and to train them. This is an empirical observation, not a theorem: the inner problem is nonconcave and PGD offers no global guarantee.

Practical analysis.

- **Step size and iterations.** Too small an α with few iterations never reaches the boundary (the bound $T \geq \epsilon/\alpha$ above). Too large an α wastes the iteration: if $\alpha \geq 2\epsilon$, then after the random start each coordinate offset lies in $[-\epsilon, \epsilon]$, the sign step moves it by $\pm\alpha$, and the projection sends it to $\pm\epsilon$. Every iterate is then a vertex of the ball, the update can no longer refine the solution, and the method degenerates into hopping among vertices. A typical CIFAR-10 setting is $\epsilon = 8/255$, $\alpha = 2/255$, $T \in [20, 100]$.
- **Loss choice.** This is the surrogate gap of Remark 2.1. Cross-entropy saturates when the model is very confident, giving tiny gradients. Margin losses on logits, such as $\max_{j \neq y} Z_j(\mathbf{x}') - Z_y(\mathbf{x}')$ (Section 4.2.1), or the difference-of-logits-ratio loss, are often stronger.

- **Fixed schedules are brittle.** APGD (**?**) adapts the step size and adds momentum, removing the hyperparameter sensitivity of PGD. Together with complementary attacks, it forms AutoAttack, the standard benchmark for reporting ℓ_∞ and ℓ_2 robustness of deterministic, differentiable models. A fixed benchmark does not replace an adaptive attack designed against a specific defense, which remains the standard for evaluating new defenses (**?**) (Chapter 6).

Reading PGD results

A PGD robust accuracy is an *upper bound* on true robust accuracy (Section 2.4). Sanity checks: accuracy should decrease monotonically with ϵ and reach zero for large ϵ; more iterations and restarts should not increase accuracy; PGD should beat FGSM; white-box should beat black-box. Violations indicate gradient masking or a bug (**?**).

4.2 Gradient-Based Minimum-Norm Attacks

The attacks of Section 4.1 report whether the model can be fooled at a given ϵ; to draw the robustness curve of Proposition 2.1 they must be rerun for every budget. We now turn to attacks on the min-norm problem (2), which return a distance instead. Section 2.2 named its obstacles: the constraint $f(\mathbf{x}+\boldsymbol{\delta}) \neq y$ is discrete and gives no gradient, the feasible set is nonconvex, and the box $\mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}$ is awkward for unconstrained gradient methods. The two attacks deal with them differently:

- **C&W** (**?**) uses the penalty form (3) with a logit margin, removes the box constraint by a change of variables, solves the resulting unconstrained problem with Adam, and selects the penalty weight per input by binary search. It exists for ℓ_0, ℓ_2 and ℓ_∞.
- **DeepFool** (**?**) keeps the constraint and replaces the classifier: at each step it linearizes all class boundaries around the current iterate and jumps to the nearest one. For a linear classifier this recovers the exact solution of (2) for ℓ_2 (compare Exercise 2 of Chapter 1).

4.2.1 Carlini and Wagner

From a hard constraint to a penalty. Replace the classification constraint by a function h such that $f(\mathbf{x}') = t$ holds whenever $h(\mathbf{x}') \leq 0$. After comparing several candidates, C&W recommend the logit margin

$$h(\mathbf{x}') = \max\Big(\max_{j \neq t} Z_j(\mathbf{x}') - Z_t(\mathbf{x}'),\ -\kappa\Big), \tag{8}$$

where $\kappa \geq 0$ is a *confidence* parameter: the attack keeps pushing until the target logit exceeds every other logit by at least κ. For the untargeted version, use $\max(Z_y - \max_{j \neq y} Z_j, -\kappa)$. Working with logits instead of softmax probabilities avoids saturation.

Removing the box constraint. Change variables to $\mathbf{w} \in \mathbb{R}^d$ with

$$\mathbf{x}' = \tfrac{1}{2}\big(\tanh(\mathbf{w}) + 1\big),$$

so that $\mathbf{x}' \in \mathcal{X}$ automatically and the problem becomes unconstrained in $\mathbf{w}$.

The ℓ_2 attack. Combining both, and using the squared norm in (3),

$$\min_{\mathbf{w}} \ \big\|\tfrac{1}{2}(\tanh(\mathbf{w}) + 1) - \mathbf{x}\big\|_2^2 + c \cdot h\big(\tfrac{1}{2}(\tanh(\mathbf{w}) + 1)\big), \tag{9}$$

solved with Adam. The constant $c > 0$ trades off distortion against success; it is chosen per input by *binary search*: increase c when the attack fails, decrease it when it succeeds, and keep the smallest successful perturbation.

Example 4.3 (The role of c and κ)**.** Apply the untargeted ℓ_2 objective to Example 2.1, ignoring the change of variables, which plays no role here because the optimum lies inside the box. With $y = 1$, $h(\mathbf{x}+\boldsymbol{\delta}) = \max\big(-m(\mathbf{x}+\boldsymbol{\delta}), -\kappa\big) = \max\big(1 - \mathbf{a}^\top\boldsymbol{\delta}, -\kappa\big)$. For a fixed value of $\mathbf{a}^\top\boldsymbol{\delta}$, $\|\boldsymbol{\delta}\|_2$ is smallest when $\boldsymbol{\delta}$ is parallel to $\mathbf{a}$, so we may write $\boldsymbol{\delta} = t\,\mathbf{a}/5$ with $t \geq 0$, and the objective becomes

$$g(t) = t^2 + c\,\max(1 - 5t,\ -\kappa).$$

Take $\kappa = 0$ first. For $t < 0.2$, $g'(t) = 2t - 5c$, which vanishes at $t = 2.5c$. If $c < 0.08$, the minimizer is $t^\star = 2.5c < 0.2$ and $m(\mathbf{x}+\boldsymbol{\delta}) = -1 + 12.5c < 0$: the penalty is too weak and the solution is not adversarial. If $c \geq 0.08$, g decreases on $[0, 0.2)$ and increases afterwards, so $t^\star = 0.2$, which is the minimal perturbation of

Example 2.1; it lies exactly on the boundary, $m = 0$, and is adversarial only through tie-breaking. The binary search on c locates the threshold 0.08: below it the attack fails, above it the solution no longer depends on c. With $\kappa > 0$ the same computation gives $t^\star = \min\big(2.5c,\ (1+\kappa)/5\big)$, so for $c \geq 0.08(1+\kappa)$ the solution is adversarial with margin κ. The confidence parameter moves the solution a distance $\kappa/\|\mathbf{a}\|_2 = \kappa/5$ past the boundary. It costs norm, and Example 6.1 shows what it buys.

ℓ_0 and ℓ_∞ variants. The ℓ_∞ norm is non-differentiable and gradient descent on it oscillates, so C&W replace it by the penalty $\sum_i \max(|\delta_i| - \tau, 0)$ and shrink τ over iterations. The ℓ_0 attack repeatedly runs the ℓ_2 attack and freezes the pixels that contributed least, shrinking the set of modifiable features.

Analysis.

- C&W produces much smaller perturbations than FGSM/PGD at a fixed success rate and was historically decisive in breaking defenses such as defensive distillation, which shrank softmax gradients but left logits informative.
- It is expensive: thousands of gradient steps per input including the binary search.
- To compare with PGD, use Proposition 2.1: count the fraction of inputs whose C&W perturbation is below ϵ.
- The margin (8) is widely reused as the objective of the budget problem inside PGD ("PGD-CW"), often outperforming cross-entropy.

4.2.2 DeepFool

C&W solves a penalized problem with thousands of gradient steps. DeepFool (**?**) instead exploits the fact that problem (2) has a closed-form solution when the classifier is linear, and applies that solution repeatedly to the local linearization of a nonlinear one. We write the derivation for ℓ_2 and state the general ℓ_p case afterwards.

Step 1: the linear binary case. Let $g(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} + b$ with $f(\mathbf{x}) = \operatorname{sign}(g(\mathbf{x}))$, $\mathbf{a} \neq \mathbf{0}$.

Proposition 4.4 (Distance to a hyperplane)**.** *The minimal ℓ_2 perturbation that changes the sign of g at $\mathbf{x}$ has norm $|g(\mathbf{x})|/\|\mathbf{a}\|_2$, and is attained at*

$$\boldsymbol{\delta}^\star = -\frac{g(\mathbf{x})}{\|\mathbf{a}\|_2^2}\,\mathbf{a}.$$

Proof. Changing the sign requires $g(\mathbf{x}+\boldsymbol{\delta}) = g(\mathbf{x}) + \mathbf{a}^\top\boldsymbol{\delta}$ to reach zero or beyond, so $|\mathbf{a}^\top\boldsymbol{\delta}| \geq |g(\mathbf{x})|$ with $\mathbf{a}^\top\boldsymbol{\delta}$ of sign opposite to $g(\mathbf{x})$. By Cauchy–Schwarz, $|\mathbf{a}^\top\boldsymbol{\delta}| \leq \|\mathbf{a}\|_2\|\boldsymbol{\delta}\|_2$, hence $\|\boldsymbol{\delta}\|_2 \geq |g(\mathbf{x})|/\|\mathbf{a}\|_2$. Equality in Cauchy–Schwarz holds when $\boldsymbol{\delta}$ is parallel to $\mathbf{a}$, which gives $\boldsymbol{\delta}^\star$, and then $g(\mathbf{x}+\boldsymbol{\delta}^\star) = 0$. □

The perturbation is the orthogonal projection of $\mathbf{x}$ onto the hyperplane $\{g = 0\}$. It is the same Hölder argument as in Proposition 4.1, read as a min-norm problem instead of a budget problem.

Step 2: the linear multiclass case. Let $Z_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$ and $f(\mathbf{x}) = y$. For each $k \neq y$ define the margin

$$m_k(\mathbf{x}) = Z_k(\mathbf{x}) - Z_y(\mathbf{x}) = \mathbf{a}_k^\top \mathbf{x} + \beta_k, \qquad \mathbf{a}_k = \mathbf{w}_k - \mathbf{w}_y,\ \ \beta_k = b_k - b_y,$$

which is negative at $\mathbf{x}$ because $f(\mathbf{x}) = y$. The region where the model predicts y is the polyhedron $P = \{\mathbf{x} : m_k(\mathbf{x}) < 0\ \forall k \neq y\}$, and its complement is the union of the half-spaces $H_k = \{m_k \geq 0\}$. Hence

$$\epsilon^\star(\mathbf{x}, y) = \min_{k \neq y} \operatorname{dist}_2(\mathbf{x}, H_k) = \min_{k \neq y} \frac{|m_k(\mathbf{x})|}{\|\mathbf{a}_k\|_2}, \tag{10}$$

using Proposition 4.4 for each k. The minimizer is the class l whose boundary is closest, and the optimal perturbation is

$$\boldsymbol{\delta}^\star = \frac{|m_l(\mathbf{x})|}{\|\mathbf{a}_l\|_2^2}\,\mathbf{a}_l, \qquad l = \arg\min_{k \neq y} \frac{|m_k(\mathbf{x})|}{\|\mathbf{a}_k\|_2}.$$

This is exact when the box constraint is inactive. The nearest boundary in terms of *logit gap* $|m_k|$ is not in general the nearest in terms of *distance*, because the denominators $\|\mathbf{a}_k\|_2$ differ.

Step 3: nonlinear classifiers. For a differentiable Z, replace m_k around the current iterate $\mathbf{x}^{(i)}$ by its first-order model

$$m_k(\mathbf{x}^{(i)} + \boldsymbol{\delta}) \approx m_k(\mathbf{x}^{(i)}) + \nabla m_k(\mathbf{x}^{(i)})^\top \boldsymbol{\delta}, \qquad \nabla m_k = \nabla Z_k - \nabla Z_y,$$

which is the linear multiclass classifier of Step 2 with $\mathbf{a}_k = \nabla m_k(\mathbf{x}^{(i)})$. One DeepFool step applies the formula above to this model:

$$l_i = \arg\min_{k \neq y} \frac{|m_k(\mathbf{x}^{(i)})|}{\|\nabla m_k(\mathbf{x}^{(i)})\|_2}, \qquad \mathbf{r}_i = \frac{|m_{l_i}(\mathbf{x}^{(i)})|}{\|\nabla m_{l_i}(\mathbf{x}^{(i)})\|_2^2} \nabla m_{l_i}(\mathbf{x}^{(i)}). \tag{11}$$

Since the linear model is only local, the iteration is repeated from the new point until the label changes. The final perturbation is the accumulated sum of the steps.

Algorithm 2 DeepFool, ℓ_2, untargeted

Require: $\mathbf{x}, y = f(\mathbf{x})$, overshoot η, maximum iterations T

$\mathbf{x}^{(0)} \leftarrow \mathbf{x}$, $\mathbf{r}_{\text{tot}} \leftarrow \mathbf{0}$
for $i = 0, \dots, T-1$ **do**
  if $f(\mathbf{x}^{(i)}) \neq y$ **then break**
  end if
  compute $m_k(\mathbf{x}^{(i)})$ and $\nabla m_k(\mathbf{x}^{(i)})$ for all $k \neq y$ ▷ $K-1$ backward passes
  compute l_i and $\mathbf{r}_i$ by Eq. (11)
  $\mathbf{r}_{\text{tot}} \leftarrow \mathbf{r}_{\text{tot}} + \mathbf{r}_i$
  $\mathbf{x}^{(i+1)} \leftarrow \text{clip}_{[0,1]}\big(\mathbf{x} + (1+\eta)\,\mathbf{r}_{\text{tot}}\big)$
end for
return $\mathbf{x}' = \mathbf{x}^{(i)}$ and $\|\mathbf{x}' - \mathbf{x}\|_2$

The overshoot $\eta > 0$ (the original paper uses $\eta = 0.02$) is needed because each step lands *on* the linearized boundary, and the true boundary is only approximated. Without it the iterate often stops at $m_l \approx 0^-$ and the label does not change. The update recomputes the point from $\mathbf{x}$ and the accumulated perturbation, so the overshoot does not compound across iterations.

General ℓ_p. The same derivation with Hölder's inequality in place of Cauchy–Schwarz gives the distance $|m_k|/\|\nabla m_k\|_q$ and the step

$$\mathbf{r}_i = \frac{|m_l(\mathbf{x}^{(i)})|}{\|\nabla m_l(\mathbf{x}^{(i)})\|_q^q} \operatorname{sign}(\nabla m_l)\,|\nabla m_l|^{q-1}.$$

For $p = \infty$ ($q = 1$) this reduces to $\mathbf{r}_i = |m_l| \operatorname{sign}(\nabla m_l)/\|\nabla m_l\|_1$, which is again the sign step of Proposition 4.1, now with a step size chosen to land on the linearized boundary instead of a fixed budget.

Example 4.4 (DeepFool on the two examples)**.** On the linear classifier of Example 2.1 the linearization is exact, and the first step of Eq. (11) is $\mathbf{r}_0 = \frac{1}{25}(3, 4) = (0.12, 0.16)$, the exact minimizer. With overshoot $\eta = 0.02$ the returned distance is $1.02 \times 0.2 = 0.204$. On the piecewise-linear model of Example 4.2, the piece active at $x = 0.5$ is $x - 0.7$, with $m = -0.2$ and slope 1, so the first step is $r_0 = 0.2$ to the right. The iterate $0.5 + 1.02 \times 0.2 = 0.704$ is misclassified and DeepFool stops, returning 0.204, whereas $\epsilon^\star = 0.15$. DeepFool found the boundary of the piece that is active at x, not the nearest boundary. This is the min-norm counterpart of the FGSM failure in Example 4.2, and it is why the returned value is only an upper bound on $\epsilon^\star$.

Analysis.

- DeepFool is exact for linear classifiers (Eq. 10) and is a fixed-point iteration of local linearizations for nonlinear ones. It has no optimality guarantee: the returned norm is an upper bound on $\epsilon^\star(\mathbf{x}, y)$, as for any attack.
- It needs no penalty weight and no binary search, and typically converges in a small number of iterations. The cost per iteration is up to $K-1$ backward passes, which can be reduced by restricting the search to the few classes with the largest logits.
- It tends to return larger perturbations than C&W on the same input, since C&W optimizes the distortion directly. Its role is a fast estimate of the distance to the boundary, for example to compute an average robustness over a dataset, and not the strongest attack for evaluating a defense.
- The box constraint is handled by clipping, which is only an approximation of the constrained problem, and the ℓ_2 projection issue of Exercise **2.** applies here as well.
- Because it linearizes the logits, it is affected by gradient masking in the same way as FGSM: if ∇m_k is uninformative, the step is unreliable.

5 Query-Based Attacks

Every attack of Section 4 uses $\nabla_{\mathbf{x}}\mathcal{L}$ or $\nabla_{\mathbf{x}}Z_k$, which requires the architecture and the weights. Deployed models are usually reachable only through an interface that returns outputs, so we now drop this assumption: the attacker cannot differentiate through f but can query it on chosen inputs. The cost of an attack is measured in queries. We distinguish what a query returns: scores (Section 5.1) or only the predicted label (Section 5.2). The information level decides which of the two problems of Section 2 remains tractable.

5.1 Score-Based Attacks

Formulation. Suppose the attacker can query $F(\mathbf{x}')$ (or the top-k scores) at any point. Then the surrogate $\mathcal{L}(\mathbf{x}', y)$, the margin $h(\mathbf{x}')$ of Eq. (8), and the penalized objective (3) can all be *evaluated*; what is missing is their gradient. Both problems therefore keep their formulation, and only the solver changes: either the gradient is estimated from function values (zeroth-order optimization) and plugged into a white-box solver, or it is avoided altogether by a search that needs only comparisons of objective values. ZOO estimates gradients for the penalized min-norm problem (9); NES, SPSA and the Square Attack solve the budget problem (1).

5.1.1 Gradient Estimation

Coordinate-wise finite differences. ZOO (**?**) solves the C&W objective (9), with the margin computed from scores, replacing each partial derivative by a symmetric difference,

$$\hat{g}_i = \frac{\mathcal{L}(\mathbf{x}' + h\mathbf{e}_i, y) - \mathcal{L}(\mathbf{x}' - h\mathbf{e}_i, y)}{2h},$$

and updating a random batch of coordinates per iteration. An exact gradient needs $2d$ queries, which is prohibitive for images; ZOO mitigates this by attacking in a downsampled space and prioritizing important coordinates.

Random-direction estimators (NES). **?** solve the budget problem with PGD in which the gradient is replaced by a natural evolution strategies estimate. They sample n directions $\mathbf{u}_i \sim \mathcal{N}(0, I)$ and estimate

$$\hat{\mathbf{g}} = \frac{1}{2n\sigma} \sum_{i=1}^{n} \big[\mathcal{L}(\mathbf{x}' + \sigma\mathbf{u}_i, y) - \mathcal{L}(\mathbf{x}' - \sigma\mathbf{u}_i, y)\big]\mathbf{u}_i, \tag{12}$$

which is an unbiased estimate of the gradient of a Gaussian-smoothed loss. Plugging $\hat{\mathbf{g}}$ into the PGD update of Section 4.1 gives an effective attack with far fewer than d queries per step, and the projection is unchanged because the feasible set $\mathcal{S}_p(\mathbf{x}, \epsilon)$ does not depend on f. The same paper handles the *partial-information* (top-k only) and *label-only* settings. SPSA (**?**) is a closely related estimator using Rademacher directions. Because it does not use backpropagated gradients, it is one of the gradient-free attacks recommended, together with a decision-based attack, as a check for gradient masking (**?**): a gradient-free attack should not succeed more often than a gradient-based one. The check is weaker for randomized models, against which gradient-free attacks tend to perform poorly.

5.1.2 Random Search

Square Attack. **?** solve the budget problem with a margin objective and no gradient estimate. The search space is motivated by Proposition 4.1: for ℓ_∞ the maximizer of a linear objective over the ball is a vertex, with every coordinate offset equal to $\pm\epsilon$, so the attack restricts itself to vertices. At each iteration a random square window is chosen and the perturbation inside it is resampled at values $\pm\epsilon$; the change is accepted only if it improves the margin objective. Window size shrinks over time. Despite its simplicity, Square Attack is among the most query-efficient score-based attacks and, because it never touches gradients, is immune to gradient masking. It is part of AutoAttack for that reason.

Analysis. Score-based attacks reveal that hiding gradients is not a defense: if scores are exposed, the objectives of both problems can be evaluated and gradients approximated. Their cost is measured in queries and depends strongly on the setting. Against standard (non-robust) classifiers, untargeted ℓ_∞ attacks such as the Square Attack often succeed with tens to a few hundred queries on average, while targeted attacks, ℓ_2 attacks at small budgets and attacks on robust models need far more. The distribution of query counts is right-skewed, so the median should be reported together with the mean. Defenses that only return labels, add noise to scores, or rate-limit queries raise the cost but do not eliminate the threat.

5.2 Decision-Based Attacks

Formulation. Score-based attacks rely on a continuous output: even with the gradient hidden, a small change of the input produces a measurable change of $\mathcal{L}$. Removing the scores removes that signal. In the hardest query setting the attacker sees only $f(\mathbf{x}')$. The budget objective then reduces to the indicator $\mathbb{1}[f(\mathbf{x}') \neq y]$, which is constant on almost all of $\mathcal{S}_p(\mathbf{x}, \epsilon)$: unless the attacker is already at an adversarial point, no query inside the ball indicates which direction to move, and finite differences are zero almost everywhere. The min-norm problem (2) is better suited to this information level. Its objective $\|\mathbf{x}' - \mathbf{x}\|_p$ is computed without the model, and the model is needed only to test feasibility, a one-bit oracle. Decision-based attacks are therefore *feasible-point methods* for (2): they start from an adversarial point, possibly far from $\mathbf{x}$, and reduce $\|\mathbf{x}' - \mathbf{x}\|_p$ while keeping $\mathbf{x}'$ adversarial. The budget question is answered through Definition 1.3: the attack succeeds at ϵ as soon as $\|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon$, and can stop there.

Boundary Attack. **?** start from a point that is already adversarial (for untargeted attacks, random noise that is misclassified; for targeted attacks, an image of the target class) and perform a random walk *along* the boundary that progressively reduces the distance to $\mathbf{x}$. Each step combines an orthogonal random perturbation (exploring along the boundary) and a small step toward $\mathbf{x}$, and is accepted only if the point remains adversarial, that is, feasible for (2). Step sizes are adapted from the acceptance rate. It needs only labels but can require tens of thousands of queries.

HopSkipJump. **?** improve query efficiency by estimating the gradient direction *at the boundary* from binary feedback. At a boundary point $\mathbf{x}_b$, define $\phi(\mathbf{z}) = +1$ if $\mathbf{z}$ is adversarial and -1 otherwise. Then

$$\widehat{\nabla} = \frac{1}{n} \sum_{i=1}^{n} \phi(\mathbf{x}_b + \sigma \mathbf{u}_i)\, \mathbf{u}_i$$

points, in expectation and for small σ, along the boundary normal. Each iteration: (i) estimate the normal, (ii) take a geometric step along it, (iii) binary-search back to the boundary on the segment toward $\mathbf{x}$. Step (iii) solves (2) exactly when restricted to that segment: $\mathbf{x}$ is infeasible, the current point is feasible, and bisection on the label finds the crossing closest to $\mathbf{x}$ up to the tolerance. It supports both ℓ_2 and ℓ_∞ and converges with far fewer queries than the Boundary Attack.

Example 5.1 (One HopSkipJump iteration on the linear classifier)**.** In Example 2.1, suppose the attacker sees only labels and starts from the adversarial point $\mathbf{x}'_0 = (1, 1)$, where $m = 3$. Along the segment $\mathbf{x} + s(\mathbf{x}'_0 - \mathbf{x})$, $s \in [0, 1]$, the margin is $-1+4s$, so bisection on the label converges to $s = 1/4$, the boundary point $\mathbf{x}_b = (0.4, 0.7)$, at distance $0.25\,\|(0.8, 0.4)\|_2 \approx 0.224$ from $\mathbf{x}$. This point is already feasible for (2), so 0.224 is an upper bound on $\epsilon^\star = 0.2$. On a linear boundary, $\phi(\mathbf{x}_b + \sigma\mathbf{u}) = \operatorname{sign}(\mathbf{a}^\top \mathbf{u})$ exactly, and for $\mathbf{u} \sim \mathcal{N}(0, I)$, $\mathbb{E}[\operatorname{sign}(\mathbf{a}^\top \mathbf{u})\, \mathbf{u}] = \sqrt{2/\pi}\, \mathbf{a}/\|\mathbf{a}\|_2$, so the estimated normal is aligned with $\mathbf{a}$ in expectation. A step of length 0.2 along the unit normal $(0.6, 0.8)$ gives $\mathbf{z} = (0.52, 0.86)$, with $m(\mathbf{z}) = 1$. On the segment from $\mathbf{x}$ to $\mathbf{z}$ the margin is $-1 + 2s$, so bisection returns $s = 1/2$, the point $(0.36, 0.73)$, at distance $0.5\,\|(0.32, 0.26)\|_2 \approx 0.206$. One iteration reduced the upper bound from 0.224 to 0.206, and further iterations approach the minimizer $(0.32, 0.76)$.

Information versus cost

White-box	exact gradients	tens of forward/backward passes
Score-based	loss values	10^1–10^4 queries
Decision-based	labels	10^3–10^5 queries
Transfer	nothing from the target per input	no per-input queries, lower success

Ranges are indicative: the lower end corresponds to untargeted attacks on standard models, the upper end to targeted attacks and robust models. Less information means higher cost and, usually, larger perturbations, but almost never complete protection.

6 Transfer Attacks

Query-based attacks pay for every attacked input, and a defender who limits or monitors queries raises that price. Transfer attacks remove the per-input cost by moving the optimization to a model the attacker owns. **?** already noted that adversarial examples crafted for one network often fool another network trained on different data or with a different architecture. **?** turned this into a practical black-box attack: train a *substitute* model by querying the target on synthetic inputs (using Jacobian-based dataset augmentation), attack the substitute with white-box methods, and submit the result to the target. The queries are spent once to build the substitute, not

per attacked input. If a surrogate is available without any access to the target (a public pretrained model, for example), no query is needed at all.

Formulation. Let $\hat{f}$ be the surrogate, with loss $\hat{\mathcal{L}}$. The attacker solves one of the two problems for $\hat{f}$ and submits the result to f. The two choices behave very differently.

- *Budget.* The solution $\hat{\mathbf{x}}'$ of $\max_{\mathbf{x}' \in \mathcal{S}_p(\mathbf{x},\epsilon)} \hat{\mathcal{L}}(\mathbf{x}', y)$ is *feasible* for the target's budget problem, because $\mathcal{S}_p(\mathbf{x}, \epsilon)$ does not depend on the model. Only the objective is wrong: $\hat{\mathbf{x}}'$ maximizes $\hat{\mathcal{L}}$, not $\mathcal{L}$, and succeeds if the two are aligned enough that $f(\hat{\mathbf{x}}') \neq y$.
- *Min-norm.* The solution of (2) for $\hat{f}$ lies, by construction, just across the boundary of $\hat{f}$. The boundary of f is elsewhere, so $\hat{\mathbf{x}}'$ is often *infeasible* for the target's min-norm problem: the minimality that makes it a good solution for $\hat{f}$ leaves no slack for the difference between the two models.

This is why transfer attacks are almost always run as budget attacks at a fixed ϵ, and why the confidence parameter κ of C&W, which forces the point a margin κ past the surrogate's boundary, improves transfer.

Example 6.1 (Transfer between two linear models)**.** Let the target be the classifier of Example 2.1: margin -1 at $\mathbf{x}$, unit normal $\mathbf{u} = \mathbf{a}/\|\mathbf{a}\|_2 = (0.6, 0.8)$, and $\epsilon_2^\star = 0.2$. Let the surrogate be linear with unit normal $\hat{\mathbf{u}}$, at angle $\theta \in [0, \pi/2)$ to $\mathbf{u}$. The ℓ_2 budget attack on the surrogate returns $\boldsymbol{\delta} = \epsilon\,\hat{\mathbf{u}}$ (the box is inactive for small ϵ), and the target margin becomes $-1 + 5\epsilon\cos\theta$. The attack transfers if and only if $\epsilon \geq 0.2/\cos\theta$. The surrogate's direction is optimal for the target only when $\theta = 0$; otherwise the attacker pays a factor $1/\cos\theta$ in budget, for example $\epsilon \geq 0.231$ for $\theta = 30°$. The min-norm attack on the surrogate returns $\boldsymbol{\delta} = \hat{\epsilon}^\star\hat{\mathbf{u}}$, where $\hat{\epsilon}^\star$ is the distance to the surrogate's boundary, and it transfers if and only if $\hat{\epsilon}^\star \geq 0.2/\cos\theta$. Whether this holds depends on a quantity unrelated to the target, and if the two boundaries are at the same distance from $\mathbf{x}$ it fails for every $\theta > 0$. The confidence parameter of Example 4.3 lengthens the surrogate perturbation by $\kappa/\|\hat{\mathbf{a}}\|_2$ and closes the gap once κ is large enough. For $\theta \geq \pi/2$ nothing transfers at any budget.

Why transfer works. Two complementary explanations:

- **Shared subspaces. ?** estimate that adversarial perturbations span a subspace of considerable dimension around each input, and that the decision boundaries of different models are close and aligned within it, so perturbations crossing one boundary tend to cross the other.
- **Shared non-robust features.** If models trained on the same distribution learn the same brittle but predictive features (**?**), a perturbation that manipulates those features affects all of them.

What makes examples transfer better. Strong white-box optimization tends to *overfit* to the surrogate: highly specific perturbations exploit idiosyncrasies of one model. Empirically:

- Untargeted examples transfer far better than targeted ones (**?**).
- Higher-confidence examples (larger κ in C&W) transfer better than minimal ones, as argued above.
- *Momentum* stabilizes the update direction and reduces overfitting to the surrogate (MI-FGSM, **?**):

$$\mathbf{m}^{(k+1)} = \mu\,\mathbf{m}^{(k)} + \frac{\nabla_{\mathbf{x}}\hat{\mathcal{L}}(\mathbf{x}^{(k)}, y)}{\|\nabla_{\mathbf{x}}\hat{\mathcal{L}}(\mathbf{x}^{(k)}, y)\|_1}, \qquad \mathbf{x}^{(k+1)} = \Pi_{\mathcal{S}_\infty(\mathbf{x},\epsilon)}\big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}(\mathbf{m}^{(k+1)})\big).$$

- *Input diversity*: applying random resizing and padding before each gradient computation acts like data augmentation for the attack (**?**).
- Attacking an *ensemble* of surrogates (next section).

Transfer and evaluation

Transfer success depends heavily on surrogate choice; low transfer success is not evidence of robustness. Conversely, if a transfer attack beats your white-box attack on a defended model, the white-box attack is failing, which is a classic sign of gradient masking.

7 Ensembles of Models

The last item of Section 6 suggested attacking several surrogates at once. Ensembles appear in this chapter in two roles: as surrogates that improve transfer, and as targets that claim robustness. In neither role do they define a new attack family or a new problem: they change the *objective* of the budget (or min-norm) problem, while the feasible set stays $\mathcal{S}_p(\mathbf{x}, \epsilon)$.

Ensembles as surrogates. **?** showed that optimizing a perturbation against several models simultaneously yields examples that transfer much better, including targeted examples, to an unseen model. For models $f_1, \dots, f_M$ with weights $w_m \geq 0$, $\sum_m w_m = 1$, the common fusion choices are

$$\begin{aligned} \text{logit fusion:} \quad & Z_{\text{ens}}(\mathbf{x}) = \textstyle\sum_m w_m Z_m(\mathbf{x}), \\ \text{probability fusion:} \quad & F_{\text{ens}}(\mathbf{x}) = \textstyle\sum_m w_m F_m(\mathbf{x}), \\ \text{loss fusion:} \quad & \mathcal{L}_{\text{ens}}(\mathbf{x}, y) = \textstyle\sum_m w_m \mathcal{L}_m(\mathbf{x}, y). \end{aligned}$$

Each defines a surrogate objective for (1). Logit fusion is usually the most effective for transfer. The intuition: a direction that fools many diverse models is more likely to exploit features shared by all models, including the unseen target.

Ensembles as defenses. An ensemble is a single differentiable function. If the defender averages logits or probabilities, a white-box attacker simply attacks the averaged output with PGD, exactly as for a single model. Problems arise only with *non-differentiable* aggregation (majority vote) or with *randomized* selection of members. In those cases:

- Replace the vote with a differentiable surrogate (average of logits or probabilities) for gradient computation; the resulting example usually fools the vote as well.
- For random member selection, solve the budget problem with the *expected* loss as objective, $\max_{\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)} \mathbb{E}_m[\mathcal{L}_m(\mathbf{x}', y)]$, i.e., Expectation over Transformation (**?**).
- For ensembles of detectors or heterogeneous defenses, attack all components jointly with a combined loss. **?** showed that combining several weak defenses does not yield a strong one under adaptive attack.

Ensemble adversarial training (**?**) is a different idea: augmenting training with adversarial examples transferred from other, fixed models. It improves robustness to *transfer* attacks specifically, not to white-box attacks on the defended model.

8 Large Language Models

Every attack so far targeted a classifier with continuous inputs and an ℓ_p threat set. Large language models (LLMs) break these assumptions in four ways, and each one forces a change in the definitions or in the algorithms.

1. **Discrete inputs.** The input is a sequence of tokens, so $\mathbf{x} + \boldsymbol{\delta}$ is not an input and gradient steps do not stay inside the domain.
2. **Structured outputs.** The model produces a sequence by sampling, not a label. "Misbehaving" is a predicate on the whole output, usually evaluated by a judge.
3. **No ℓ_p ball.** The admissible set is defined by what the attacker controls (a suffix, a document in a retrieval index, a conversation turn) or by preserving the meaning of a request, not by a distance.
4. **Data and instructions share one channel.** Everything is text in one context window, so an attacker who controls any part of the context can issue instructions (**?**). This is the structural analogue of SQL injection.

The general threat-set formulation of Section 2.5 absorbs the third point, and the three-axis taxonomy of Section 3 still applies. What changes is the threat set, the success predicate, the surrogate loss and the solver. Section 8.1 adapts the definitions and Section 8.2 the problem formulation; Sections 8.3 and 8.4 revisit the white-box and black-box settings; Section 8.5 treats prompt injection, which has no direct counterpart for classifiers, and Section 8.6 discusses defenses.

8.1 Threat Model

Let $\mathcal{V}$ be the vocabulary, $N = |\mathcal{V}|$, and let $\mathbf{x} \in \mathcal{V}^n$ be a token sequence. The model defines an autoregressive distribution over responses $\mathbf{r} = (r_1, \dots, r_m)$,

$$p(\mathbf{r} \mid \mathbf{x}) = \prod_{s=1}^{m} p(r_s \mid \mathbf{x}, r_{<s}).$$

A *policy* is a predicate $J(\mathbf{x}, \mathbf{r}) \in \{0, 1\}$ that equals 1 when the response is unacceptable for that prompt (a harmful completion, a leaked system prompt, an action the user did not request). In practice J is a human, a classifier, or another LLM, and this has consequences for evaluation (see the warning box at the end of the section).

Definition 8.1 (Evasion against an LLM)**.** Let q be a request that the model should not fulfill (so $J(q, \mathbf{r}) = 1$ for compliant responses). Let $\mathcal{T}(q) \subseteq \mathcal{V}^*$ be a set of admissible transformations of q (the threat set). An attack returns $\mathbf{x}' \in \mathcal{T}(q)$, and it *succeeds* if

$$\Pr_{\mathbf{r} \sim p(\cdot|\mathbf{x}')} \big[J(q, \mathbf{r}) = 1 \big] \geq \tau$$

for a chosen threshold τ (for example, τ small for "at least one sample succeeds" and close to 1 for "reliably").

This is Definition 1.1 with three substitutions: the admissible set $\mathcal{S}_p(\mathbf{x}, \epsilon)$ becomes $\mathcal{T}(q)$, as in Section 2.5; the condition $f(\mathbf{x}') \neq y$ becomes the predicate J; and because the output is sampled, success is a probability compared with a threshold. The common choices of $\mathcal{T}$ are:

- **Suffix (or prefix) of length k:** $\mathcal{T}_k(q) = \{q \oplus \mathbf{s} : \mathbf{s} \in \mathcal{V}^k\}$, where $\oplus$ is concatenation. The budget is the length k, and the sets are nested if shorter suffixes are padded with a neutral token.
- **Intent-preserving rewrites:** $\mathcal{T}(q) = \{\mathbf{x} : \ \mathbf{x}$ asks for the same thing as $q\}$. This is the analogue of "imperceptible", and unlike $\|\boldsymbol{\delta}\|_p \leq \epsilon$ it cannot be checked by computing a norm. Whether a rewrite preserves intent is itself a judgment.
- **Attacker-controlled context:** the attacker does not write the user request at all. They control a document d that the application places into the context (indirect prompt injection, Section 8.5).

Two things from the classifier setting disappear. There is no clean pair $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$ to start from, and the ASR is a rate under a judge, not a rate of label flips.

8.2 Loss Function and Problem Formulation

The indicator J is not differentiable and is expensive to query, so, as in Remark 2.1, we replace it by a surrogate. The standard choice is the likelihood of an *affirmative prefix* $\mathbf{r}^\star$ such as "Sure, here is…", which models that comply usually begin with. For the suffix threat set,

$$\mathcal{L}(\mathbf{s}) = -\log p(\mathbf{r}^\star \mid q \oplus \mathbf{s}) = \sum_{s=1}^{m^\star} -\log p\big(r_s^\star \mid q \oplus \mathbf{s},\, r_{<s}^\star\big), \tag{13}$$

where the equality is the chain rule of probability. The budget problem (1) becomes the discrete problem

$$\min_{\mathbf{s} \in \mathcal{V}^k} \mathcal{L}(\mathbf{s}), \tag{14}$$

a minimization because $\mathcal{L}$ is now the loss of the *target* output, exactly as in the targeted version of Definition 2.1. It has N^k candidates (for $N \approx 10^5$ and $k = 20$, far too many to enumerate), no continuous structure, and a surrogate gap: a response that begins with the affirmative prefix may still refuse afterwards, and a successful response may not begin with it. This is the gap of Remark 2.1, now with a more fragile surrogate.

The min-cost analogue of (2) also exists: minimize the suffix length k, or the number of edited tokens of q, subject to success under J. It is the natural view for text *classifiers*, where word-substitution attacks report the fraction of words changed. For jailbreaks it is rarely used, because each feasibility check requires sampling responses and running the judge, so the budget form with a fixed k dominates the literature.

Linearizing a discrete problem. Let $W_E \in \mathbb{R}^{N \times h}$ be the embedding matrix. The model receives the embedding of token s_i, which is $\mathbf{e}_{s_i}^\top W_E$ for the one-hot vector $\mathbf{e}_{s_i} \in \{0, 1\}^N$. Treat the one-hot rows as real vectors, so that $\mathcal{L}$ becomes a differentiable function of $\mathbf{e}_i \in \mathbb{R}^N$ for each suffix position i, and let $\mathbf{g}_i = \nabla_{\mathbf{e}_i} \mathcal{L} \in \mathbb{R}^N$. The Taylor expansion of Eq. (4) applies verbatim with $\boldsymbol{\delta} = \mathbf{e}_w - \mathbf{e}_v$, the change caused by replacing token v at position i by token w:

$$\mathcal{L}(\mathbf{s}_{i \leftarrow w}) \ \approx \ \mathcal{L}(\mathbf{s}) + (\mathbf{e}_w - \mathbf{e}_v)^\top \mathbf{g}_i = \mathcal{L}(\mathbf{s}) + g_{i,w} - g_{i,v}.$$

This is the first-order estimate used by HotFlip (**?**), and it means that all N possible replacements at position i are scored with a single backward pass.

Proposition 8.1 (Best swap under the linear model)**.** *Among all convex combinations $\boldsymbol{\lambda}$ of one-hot vectors (the simplex), $\min_{\boldsymbol{\lambda}} \boldsymbol{\lambda}^\top \mathbf{g}_i = \min_w g_{i,w}$, and the minimum is attained at the vertex $\mathbf{e}_{w^\star}$ with $w^\star = \arg\min_w g_{i,w}$. Hence the best replacement at position i under the linear model is $w^\star$, and relaxing the discrete constraint to the simplex loses nothing.*

Proof. For $\boldsymbol{\lambda} \geq 0$ with $\sum_w \lambda_w = 1$, $\boldsymbol{\lambda}^\top \mathbf{g}_i = \sum_w \lambda_w g_{i,w} \geq \sum_w \lambda_w \min_{w'} g_{i,w'} = \min_{w'} g_{i,w'}$, with equality for the vertex at the minimizing coordinate. □

The structure matches the ℓ_1 case of Exercise **1.**: a linear objective over a set whose extreme points are coordinate vectors is optimized by placing all the mass on one coordinate. The difference from ℓ_∞ FGSM is that there is no sign step spread across coordinates, only a choice of one token.

Why the linear model can only rank. In the classifier case the linearization error shrinks with the step, $O(\|\boldsymbol{\delta}\|^2)$, and BIM exploits this by taking small steps. Here no step is small: $\|\mathbf{e}_w - \mathbf{e}_v\|_2^2 = 2$ for every swap. If the relaxed loss has a β-Lipschitz gradient, the bound derived for FGSM gives $|R| \le \frac{\beta}{2} \cdot 2 = \beta$ for every swap, a fixed error that does not vanish as the attack proceeds. The estimate therefore cannot be trusted to predict the loss after a swap. It can still be used as a heuristic to *propose* candidates, provided each candidate is checked with the true loss. This is the design principle of the next algorithm. The bound itself assumes smoothness that Transformers do not satisfy globally, so we read it as intuition.

8.3 Greedy Coordinate Gradient

The Greedy Coordinate Gradient attack (GCG) of **?** solves (14) by combining the proposal and the check. At each iteration it (i) computes $\mathbf{g}_i$ for all suffix positions with one backward pass, (ii) keeps for each position the K tokens with the most negative gradient entries, (iii) samples B candidate suffixes, each obtained by replacing one random position by one of its top-K tokens, (iv) evaluates the true loss $\mathcal{L}$ of every candidate with forward passes only, and (v) moves to the best candidate.

Algorithm 3 GCG, suffix of length k

Require: request q, target prefix $\mathbf{r}^\star$, suffix length k, iterations T, top-K, batch B

initialize $\mathbf{s} \in \mathcal{V}^k$ (for example, a repeated filler token)
for $j = 1, \dots, T$ **do**
  $\mathbf{g}_i \leftarrow \nabla_{\mathbf{e}_i} \mathcal{L}(\mathbf{s})$ for $i = 1, \dots, k$ ▷ one forward and backward pass
  $\mathcal{X}_i \leftarrow$ the K tokens with smallest $g_{i,w}$
  for $b = 1, \dots, B$ **do**
    draw i uniformly in $\{1, \dots, k\}$ and w uniformly in $\mathcal{X}_i$; set $\mathbf{s}^{(b)} \leftarrow \mathbf{s}_{i \leftarrow w}$
  end for
  $\mathbf{s} \leftarrow \arg\min_b \mathcal{L}(\mathbf{s}^{(b)})$ ▷ B forward passes
end for
return $\mathbf{s}$

Universal and transferable suffixes. Replacing $\mathcal{L}$ in Eq. (13) by a sum over several requests q_j and several open models m,

$$\mathcal{L}_{\text{ens}}(\mathbf{s}) = \sum_m \sum_j -\log p_m(\mathbf{r}_j^\star \mid q_j \oplus \mathbf{s}),$$

is exactly the loss fusion of Section 7. The optimized suffix is then *universal* (one string works for many requests) and, as in Section 6, an ensemble of surrogates improves transfer to models the attacker cannot differentiate through. **?** report transfer of such suffixes to closed-source models, but how well it works depends on the target and has changed as models were updated.

Embedding-space attacks. If the attacker has the weights, nothing prevents running PGD directly on the continuous embeddings of the suffix, which is the classifier setting of Section 4.1. This is an upper bound on what token-space attacks can achieve, but it is not a realistic threat model for a user who can only submit text, because an arbitrary embedding vector need not correspond to any token: the threat set has been enlarged beyond $\mathcal{T}_k(q)$. The result of **?** that continuous inputs (images) in multimodal models are much easier to attack than text is the same phenomenon.

8.4 Black-Box Attacks

Without the weights, the attacker loses the gradient ranking that drives GCG. The chapter's information axis carries over directly, and so does the choice of problem: every attack below solves the budget form with a fixed threat set.

Score-based. If the API returns log-probabilities, the surrogate of Eq. (13) can be evaluated without gradients. Random search then does what Square Attack does for images: mutate a few random suffix tokens and keep the change only if the log-probability of the affirmative prefix increases. **?** showed that this simple adaptive strategy reaches high success rates against several safety-aligned models. Restricting the logit bias, temperature or returned log-probabilities raises the cost, as for score-based attacks on classifiers.

Judge-based (the decision-based analogue). When only the response text is visible, the only feedback is the verdict of a judge, the analogue of a label. PAIR (**?**) uses an *attacker LLM* that proposes a rewrite of the request within the intent-preserving threat set, queries the target, receives the judge's score, and refines the rewrite in the next round. Like the Boundary Attack, it needs no gradients and no scores, and its cost is measured in queries. Its authors report that it often succeeds in fewer than twenty queries against weakly aligned targets, and that it is much less effective against models with strong safety training. Unlike the Boundary Attack, it does not solve a min-cost problem: there is no distance to reduce once the rewrite succeeds. The rewrites are readable text, which matters for the defenses below. Multi-turn variants such as Crescendo spread the attack over a conversation of individually benign turns (**?**).

Hand-crafted attacks (no optimization). **?** explain many manual jailbreaks by two failure modes. *Competing objectives*: the model is trained to be helpful and to follow instructions, and the attack makes compliance win (refusal suppression, role-play, forcing an affirmative start). *Mismatched generalization*: pretraining covers a wider distribution $\mathcal{D}_{\text{pre}}$ than safety training $\mathcal{D}_{\text{safe}}$, and an input in $\mathcal{D}_{\text{pre}} \setminus \mathcal{D}_{\text{safe}}$ (Base64, ROT13, rare languages, payload splitting) is understood by the model but not covered by its refusal behavior. Neither mechanism requires a gradient, and the second one has a simple explanation in terms of the coverage of the training distribution, in the spirit of Section 6.

Attack	Information	Problem and surrogate	Solver
GCG	gradients (open weights)	budget (suffix length k), NLL of affirmative prefix	gradient-ranked candidates, forward-pass selection
Embedding-space PGD	gradients (open weights)	budget in embedding space, same NLL	PGD (not a text-only threat)
Random search on log-probs	scores	budget, log-probability of prefix	random token mutation, accept if better
PAIR, TAP-style	labels (judge verdict)	success of a rewrite under a judge	attacker LLM with feedback
Role-play, encodings	none	none (hand-crafted)	human prior about training gaps
Universal suffix, transfer	none (per request)	budget, summed over models and requests	GCG on an ensemble of open models

Table 3: LLM evasion attacks along the axes of Table 2. Entries summarize the cited papers.

8.5 Prompt Injection

The attacks above assume that the attacker writes the prompt. The third threat set of Section 8.1, attacker-controlled context, removes that assumption. In a *direct* prompt injection the attacker is the user and tries to override the developer's system prompt. In an *indirect* injection the attacker controls a document d (a web page, an email, a retrieved passage) that the application inserts into the context together with the user's request u and the system prompt $\mathbf{z}$, so the model receives a single sequence $\mathbf{z} \oplus u \oplus d$ (**??**). The attacker's goal is a behavior b that neither $\mathbf{z}$ nor u asked for (exfiltrating data to a URL, calling a tool, suppressing a source). In the terms of Section 2.5, the threat set is $\{d : \text{the attacker can place } d \text{ in the context}\}$, which is limited by access, not by distance, and the budget problem is to make $\Pr[\text{behavior } b \mid \mathbf{z} \oplus u \oplus d]$ large over that set. The same optimization techniques apply to d (for instance, optimized "execution triggers" reported by **?**), and the attack can persist through retrieval pipelines.

The root cause is architectural: the model has no reliable mechanism to distinguish instructions from data, because both are tokens in the same sequence. For this reason, defenses that only modify the model reduce but do not remove the risk, and a common recommendation is to design the application on the assumption that the model may follow injected instructions, by limiting its permissions and the data it can reach (**?**).

8.6 Defenses

The defenses below are the LLM counterparts of ideas already met for classifiers, and each has a limitation already seen in that setting.

- **Perplexity filtering.** GCG suffixes are unnatural token strings with high perplexity under a language model, and a filter that rejects prompts above a threshold detects them (**?**). In the terms of Section 2.5, the filter shrinks the threat set to low-perplexity strings. This defends against that attack and not against

PAIR-style rewrites, which are fluent. An adaptive attacker adds a fluency term to Eq. (13), trading attack strength for readability.

- **Randomized perturbation (SmoothLLM). ?** query the model on several randomly perturbed copies of the prompt and aggregate the outputs. This is motivated by randomized smoothing (Section 2.4): optimized suffixes are brittle, and character-level noise breaks them while leaving benign prompts mostly intact. It costs several queries per prompt, and the guarantee is not the certified radius of the classifier setting.
- **Training-time defenses.** Safety training, refusal training and adversarial training on attack prompts raise the cost of attacks, but recent evaluations still find current models vulnerable to direct prompt injection (**?**). Adversarial training against GCG-style suffixes faces the same difficulty as in Chapter 1: the inner maximization is expensive and the threat set is open-ended.

Evaluating LLM attacks and defenses

- **The judge is part of the threat model.** The ASR depends on who decides J. Keyword matching on refusals produces both false successes (a hedged answer without a refusal phrase) and false failures. State the judge and validate it on a labeled sample.
- **The surrogate is not the objective.** Low $\mathcal{L}(\mathbf{s})$ means the model starts with the affirmative prefix, not that the full response is harmful. Report success under J, not the loss.
- **Randomness and budget.** Sampling makes success probabilistic (the threshold τ of Definition 8.1). Report the number of samples, the number of queries, and the computational budget of the attack, since a weak attack gives a loose upper bound on robustness exactly as in Section 2.4.
- **Adaptive attacks.** A defense evaluated only against attacks that were not designed for it (a perplexity filter tested with plain GCG) says little. Follow the checklist of **?** and **?**: attack the defense, not the base model.

9 Implementation

We return to image classifiers. The following PyTorch code implements one attack per problem and information level, in its simplest form: FGSM and PGD for the budget problem with gradients, C&W for the min-norm problem, NES-PGD for the budget problem with scores, and PGD on a logit ensemble for transfer. It assumes a model `model` in `eval()` mode that receives images in $[0, 1]$ (place any normalization inside the model), and batches `x`, `y`. Any CIFAR-10 classifier will do; pretrained robust and non-robust models are available in RobustBench.

9.1 FGSM and PGD

```
import torch
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def margin_loss(logits, y):
    # max_{j != y} Z_j - Z_y   (positive => misclassified)
    true = logits.gather(1, y[:, None]).squeeze(1)
    other = logits.clone()
    other.scatter_(1, y[:, None], -float("inf"))
    return other.max(1).values - true

def pgd_linf(model, x, y, eps, alpha, steps, restarts=1, loss="ce"):
    best = x.clone()
    fooled = torch.zeros(len(x), dtype=torch.bool, device=x.device)
    for _ in range(restarts):
        delta = torch.empty_like(x).uniform_(-eps, eps)
        xa = (x + delta).clamp(0, 1).detach()
        for _ in range(steps):
            xa.requires_grad_(True)
            logits = model(xa)
            l = F.cross_entropy(logits, y) if loss == "ce" \
                else margin_loss(logits, y).sum()
            grad, = torch.autograd.grad(l, xa)
            xa = xa.detach() + alpha * grad.sign()
            xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
```

```
        with torch.no_grad():
            succ = model(xa).argmax(1) != y
        new = succ & ~fooled
        best[new] = xa[new]
        fooled |= succ
    return best, fooled
```

9.2 Carlini–Wagner ℓ_2

A single value of c is used; the binary search of Section 4.2.1 is left as an exercise in the suggested experiments.

```
def cw_l2(model, x, y, c=1.0, kappa=0.0, steps=200, lr=1e-2):
    # change of variables: x' = (tanh(w) + 1) / 2
    w = torch.atanh((x * 2 - 1).clamp(-1 + 1e-6, 1 - 1e-6)).detach()
    w.requires_grad_(True)
    opt = torch.optim.Adam([w], lr=lr)
    for _ in range(steps):
        xa = (torch.tanh(w) + 1) / 2
        logits = model(xa)
        true = logits.gather(1, y[:, None]).squeeze(1)
        other = logits.clone()
        other.scatter_(1, y[:, None], -float("inf"))
        h = torch.clamp(true - other.max(1).values, min=-kappa)
        dist = ((xa - x) ** 2).flatten(1).sum(1)
        loss = (dist + c * h).sum()
        opt.zero_grad(); loss.backward(); opt.step()
    return ((torch.tanh(w) + 1) / 2).detach()
```

9.3 NES with PGD

```
@torch.no_grad()
def nes_grad(model, x, y, sigma=1e-3, n=50):
    g = torch.zeros_like(x)
    for _ in range(n // 2):
        u = torch.randn_like(x)
        lp = F.cross_entropy(model(x + sigma * u), y, reduction="none")
        lm = F.cross_entropy(model(x - sigma * u), y, reduction="none")
        g += (lp - lm).view(-1, *[1] * (x.dim() - 1)) * u
    return g / (n * sigma)   # n queries per estimate

def nes_pgd(model, x, y, eps, alpha, steps, **kw):
    xa = x.clone()
    for _ in range(steps):
        g = nes_grad(model, xa, y, **kw)
        xa = xa + alpha * g.sign()
        xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
    return xa
```

9.4 Transfer from an Ensemble

```
class LogitEnsemble(torch.nn.Module):
    def __init__(self, models, weights=None):
        super().__init__()
        self.models = torch.nn.ModuleList(models)
        self.w = weights or [1 / len(models)] * len(models)
    def forward(self, x):
        return sum(w * m(x) for w, m in zip(self.w, self.models))

# craft on surrogates, evaluate on the held-out target
surrogate = LogitEnsemble([resnet18, vgg16, densenet121]).eval()
x_adv, _ = pgd_linf(surrogate, x, y, eps=8/255, alpha=2/255, steps=20)
transfer_rate = (target(x_adv).argmax(1) != y).float().mean()
```

9.5 Suggested Experiments

1. On a standard (non-robust) CIFAR-10 model, plot accuracy versus $\epsilon \in \{0, 1, 2, 4, 8\}/255$ for FGSM, PGD-20 with cross-entropy and PGD-20 with the margin loss. Repeat on a robust model from RobustBench. Which attack is strongest, and does the ranking change?

2. For PGD, vary α and the number of steps with fixed $\epsilon = 8/255$. Identify a configuration that underestimates attack strength and explain why.
3. Run C&W ℓ_2 with binary search over c on 100 test images. Plot the empirical survival function of the returned distances (Proposition 2.1) and compare its value at $\epsilon_2 = 0.5$ with the robust accuracy given by PGD-ℓ_2 at that budget.
4. Implement NES-PGD and plot success rate versus number of queries. How many queries are needed to match 90% of white-box PGD success?
5. Craft examples on a single surrogate and on a three-model logit ensemble. Measure transfer to a held-out architecture, with and without momentum (MI-FGSM). Report untargeted and targeted transfer rates.
6. Implement the margin of Example 4.2 as a small ReLU network. Verify that FGSM and PGD started at x fail at $\epsilon = 0.18$, estimate the success probability of PGD with one random start, and compare it with $2/9$.

Implementation pitfalls

- Keep the model in `eval()` mode; batch normalization in training mode changes outputs per batch.
- Put normalization inside the model so that ϵ is expressed in pixel space.
- Evaluate only on initially correctly classified points, or report clean accuracy alongside robust accuracy.
- Check that $\|\mathbf{x}' - \mathbf{x}\|_\infty \le \epsilon$ and $\mathbf{x}' \in [0,1]^d$ after the attack. Bugs here silently inflate success rates.

Exercises

Problem Formulation and White-Box Attacks

1. Derive the FGM update for ℓ_1, the case $p = 1$ left open in Proposition 4.1: use the upper bound with $q = \infty$ to find the maximizer. Why is the ℓ_1 steepest ascent step a poor attack in practice?
2. Show that for ℓ_2, rescaling onto the ball and then clipping to $[0,1]^d$ does not in general give the Euclidean projection onto $\mathcal{S}_2(\mathbf{x}, \epsilon)$. (Hint: try $d = 2$, $\mathbf{x} = (0,0)$, $\epsilon = 1$, $\mathbf{z} = (-1, 2)$.) Why does the argument of Proposition 4.2 not carry over?
3. Explain why cross-entropy may yield vanishing gradients for highly confident predictions, while the margin loss of Eq. (8) does not.
4. Give an example of a classifier and a point $\mathbf{x}$ for which $\epsilon^\star(\mathbf{x}, y) = \epsilon$ and no adversarial example exists at budget ϵ, and another for which one does. What does this say about the boundary case of Proposition 2.1?
5. Write the budget and min-cost problems for the rotation threat model of Table 1, with $\mathcal{T}_\epsilon(\mathbf{x}) = \{\tau_\phi(\mathbf{x}) : |\phi| \le \epsilon\}$. Which of the three advantages of ℓ_p balls listed in Section 2.5 survive?

Query-Based Attacks, Transfer and Ensembles

6. Show that Eq. (12) is an unbiased estimator of $\nabla_{\mathbf{x}} \mathbb{E}_{\mathbf{u} \sim \mathcal{N}(0,I)}[\mathcal{L}(\mathbf{x} + \sigma \mathbf{u}, y)]$. (Hint: Stein's lemma.)
7. Explain why decision-based attacks solve the min-norm problem and not the budget problem. Could a decision-based attack solve the budget problem if it were given a starting point that is already adversarial and inside $\mathcal{S}_p(\mathbf{x}, \epsilon)$?
8. A defender uses a majority vote over five independently trained networks and reports that PGD on each individual member fails to fool the vote. Describe a correct adaptive attack.
9. Classify each of the following along the three axes of Table 2 (information level, problem, solver): PGD with the margin loss, NES-PGD, HopSkipJump, MI-FGSM on a three-model ensemble. Which pairs differ in exactly one axis?

Large Language Models

10. Verify the equality in Eq. (13) from the chain rule, and explain why $\mathcal{L}$ can be evaluated for all $m^\star$ positions with a single forward pass of the sequence $q \oplus \mathbf{s} \oplus \mathbf{r}^\star$ (teacher forcing).
11. Show that the first-order estimate for a swap uses $g_{i,w} - g_{i,v}$, and explain why the term $g_{i,v}$ can be dropped when ranking replacements w at a fixed position i.
12. In GCG, the cost of one iteration is one backward pass plus B forward passes. Compare this with the cost of one PGD step and of one NES-PGD step on a classifier. When is it worth increasing B rather than K?
13. Design an adaptive attack against a perplexity filter with threshold θ: write the modified objective, and say what happens to the attack success as θ decreases.
14. Classify the following along the axes of Table 2: GCG on an ensemble of open models evaluated on a closed model, random search on log-probabilities, PAIR. For each, say which classifier-setting attack it most resembles and which assumption of that attack fails for LLMs.